

When and why PINNs fail to train: A neural tangent kernel perspective

Sifan Wang^a, Xinling Yu^a, Paris Perdikaris^{b,*}

^a Graduate Group in Applied Mathematics and Computational Science, University of Pennsylvania, Philadelphia, PA 19104, United States of America

^b Department of Mechanical Engineering and Applied Mechanics, University of Pennsylvania, Philadelphia, PA 19104, United States of America

ARTICLE INFO

Article history:

Available online 11 October 2021

Keywords:

Physics-informed neural networks

Spectral bias

Multi-task learning

Gradient descent

Scientific machine learning

ABSTRACT

Physics-informed neural networks (PINNs) have lately received great attention thanks to their flexibility in tackling a wide range of forward and inverse problems involving partial differential equations. However, despite their noticeable empirical success, little is known about how such constrained neural networks behave during their training via gradient descent. More importantly, even less is known about why such models sometimes fail to train at all. In this work, we aim to investigate these questions through the lens of the Neural Tangent Kernel (NTK); a kernel that captures the behavior of fully-connected neural networks in the infinite width limit during training via gradient descent. Specifically, we derive the NTK of PINNs and prove that, under appropriate conditions, it converges to a deterministic kernel that stays constant during training in the infinite-width limit. This allows us to analyze the training dynamics of PINNs through the lens of their limiting NTK and find a remarkable discrepancy in the convergence rate of the different loss components contributing to the total training error. To address this fundamental pathology, we propose a novel gradient descent algorithm that utilizes the eigenvalues of the NTK to adaptively calibrate the convergence rate of the total training error. Finally, we perform a series of numerical experiments to verify the correctness of our theory and the practical effectiveness of the proposed algorithms. The data and code accompanying this manuscript are publicly available at <https://github.com/PredictiveIntelligenceLab/PINNsNTK>.

© 2021 Elsevier Inc. All rights reserved.

1. Introduction

Thanks to the approximation capabilities of neural networks, physics-informed neural networks (PINNs) have already led to a series of remarkable results across a range of problems in computational science and engineering, including fluids mechanics [1–5], bio-engineering [6,7], meta-material design [8–10], free boundary problems [11], Bayesian networks and uncertainty quantification [12–16], high-dimensional PDEs [17,18], stochastic differential equations [19], fractional differential equations [20,21], and beyond [22–25]. However, PINNs using fully connected architectures often fail to achieve stable training and produce accurate predictions, especially when the underlying PDE solutions contain high-frequencies or multi-scale features [26,13,27]. Recent work by Wang et al. [28] attributed this pathological behavior to multi-scale interactions

* Corresponding author.

E-mail addresses: sifanw@sas.upenn.edu (S. Wang), xlyu@sas.upenn.edu (X. Yu), pgp@seas.upenn.edu (P. Perdikaris).

between different terms in the PINNs loss function, ultimately leading to stiffness in the gradient flow dynamics, which, consequently, introduces stringent stability requirements on the learning rate. To mitigate this pathology, Wang et al. [28] proposed an empirical learning-rate annealing scheme that utilizes the back-propagated gradient statistics during training to adaptively assign importance weights to different terms in a PINNs loss function, with the goal of balancing the magnitudes of the back-propagated gradients. Although this approach was demonstrated to produce significant and consistent improvements in the trainability and accuracy of PINNs, the fundamental reasons behind the practical difficulties of training fully-connected PINNs still remain unclear [26].

Parallel to the development of PINNs, recent investigations have shed light into the representation shortcomings and training deficiencies of fully-connected neural networks. Specifically, it has been shown that conventional fully-connected architectures – such as the ones typically used in PINNs – suffer from “spectral bias” and are incapable of learning functions with high frequencies, both in theory and in practice [29–32]. These observations are rigorously grounded by the newly developed neural tangent kernel theory [33,34] that, by exploring the connection between deep neural networks and kernel regression methods, elucidates the training dynamics of deep learning models. Specifically, the original work of Jacot et al. [33] proved that, at initialization, fully-connected networks are equivalent to Gaussian processes in the infinite-width limit [35–37], while the evolution of a infinite-width network during training can also be described by another kernel, the so-called Neural Tangent Kernel (NTK) [33]. Remarkably, this function-space viewpoint allows us then to rigorously analyze the training convergence of deep neural networks by examining the spectral properties of their limiting NTK [33,34,32].

Drawing motivation from the aforementioned developments, this work sets sail into investigating the training dynamics of PINNs. To this end, we rigorously study fully-connected PINNs models through the lens of their neural tangent kernel, and produce novel insights into when and why such models can be effectively trained, or not. Specifically, our main contributions can be summarized into the following points:

- We prove that fully-connected PINNs converge to Gaussian processes at the infinite width limit for linear PDEs.
- We derive the neural tangent kernel (NTK) of PINNs and prove that, under suitable assumptions, it converges to a deterministic kernel and remains constant during training via gradient descent with an infinitesimally small learning rate.
- We show how the convergence rate of the total training error of a PINNs model can be analyzed in terms of the spectrum of its NTK at initialization.
- We show that fully-connected PINNs not only suffer from spectral bias, but also from a remarkable discrepancy of convergence rate in the different components of their loss function.
- We propose a novel adaptive training strategy for resolving this pathological convergence behavior, and significantly enhance the trainability and predictive accuracy of PINNs.

Taken together, these developments provide a novel path into analyzing the convergence of PINNs, and enable the design of novel training algorithms that can significantly improve their trainability, accuracy and robustness.

This paper is organized as follows. In section 2, we present a brief overview of fully-connected neural networks and their behavior in the infinite-width limit following the original formulation of Jacot et al. [33]. Next, we derive the NTK of PINNs in a general setting and prove that, under suitable assumptions, it converges to a deterministic kernel and remains constant during training via gradient descent with an infinitesimally small learning rate, see section 3, 4. Furthermore, in section 5 we analyze the training dynamics of PINNs, demonstrate that PINNs models suffer from spectral bias, and then propose a novel algorithm to improve PINNs’ performance in practice. Finally we carry out a series of numerical experiments to verify the developed NTK theory and validate the effectiveness of the proposed algorithm.

2. Infinitely wide neural networks

In this section, we revisit the definition of fully-connected neural networks and investigate their behavior under the infinite-width limit. Let us start by formally defining the forward pass of a scalar valued fully-connected network with L hidden layers, with the input and output dimensions denoted as $d_0 = d$, and $d_{L+1} = 1$, respectively. For inputs $\mathbf{x} \in \mathbb{R}^d$ we also denote the input layer of the network as $\mathbf{f}^{(0)}(\mathbf{x}) = \mathbf{x}$ for convenience. Then a fully-connected neural network with L hidden layers is defined recursively as

$$\mathbf{g}^{(h)}(\mathbf{x}) = \frac{1}{\sqrt{d_h}} \mathbf{W}^{(h)} \cdot \mathbf{f}^{(h)} + \mathbf{b}^{(h)} \in \mathbb{R}^{d_{h+1}}, \quad (2.1)$$

$$\mathbf{f}^{(h+1)}(\mathbf{x}) = \sigma(\mathbf{g}^{(h)}(\mathbf{x})), \quad (2.2)$$

for $h = 0, 1, \dots, L-1$, where $\mathbf{W}^{(h)} \in \mathbb{R}^{d_{h+1} \times d_h}$ are weight matrices and $\mathbf{b}^{(h)} \in \mathbb{R}^{d_{h+1}}$ are bias vectors in the h -th hidden layer, and $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ is a coordinate-wise smooth activation function. The final output of the neural network is given by

$$f(\mathbf{x}; \boldsymbol{\theta}) = \frac{1}{\sqrt{d_L}} \mathbf{W}_L^{(L)} \cdot \mathbf{f}^{(L)} + \mathbf{b}^{(L)}, \quad (2.3)$$

where $\mathbf{W}^{(L)} \in \mathbb{R}^{1 \times d_L}$ and $\mathbf{b}^{(L)} \in \mathbb{R}$ are the weight and bias parameters of the last layer. Here, $\theta = \{\mathbf{W}^{(0)}, \mathbf{b}^{(0)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\}$ represents all parameters of the neural network. Such a parameterization is known as the “NTK parameterization” following the original work of Jacot et al. [33]. We remark that the scaling factors $\frac{1}{\sqrt{d_h}}$ are key to obtaining a consistent asymptotic behavior of neural networks as the widths of the hidden layers $d_1, d_2, \dots, d_h$ grow to infinity.

We initialize all the weights and biases to be independent and identically distributed (i.i.d.) as standard normal distribution $\mathcal{N}(0, 1)$ random variables, and consider the sequential limit of hidden widths $d_1, d_2, \dots, d_L \rightarrow \infty$. As described in [33,36,35], all coordinates of $\mathbf{f}^{(h)}$ at each hidden layer asymptotically converge to an i.i.d. centered Gaussian process with covariance function $\Sigma^{h-1} : \mathbb{R}^{d_{h-1}} \times \mathbb{R}^{d_{h-1}} \rightarrow \mathbb{R}$ defined recursively as

$$\begin{aligned}\Sigma^{(0)}(\mathbf{x}, \mathbf{x}') &= \mathbf{x}^T \mathbf{x}' + 1, \\ \Lambda^{(h)}(\mathbf{x}, \mathbf{x}') &= \begin{pmatrix} \Sigma^{(h-1)}(\mathbf{x}, \mathbf{x}) & \Sigma^{(h-1)}(\mathbf{x}, \mathbf{x}') \\ \Sigma^{(h-1)}(\mathbf{x}', \mathbf{x}) & \Sigma^{(h-1)}(\mathbf{x}', \mathbf{x}') \end{pmatrix} \in \mathbb{R}^{2 \times 2}, \\ \Sigma^{(h)}(\mathbf{x}, \mathbf{x}') &= \mathbb{E}_{(u, v) \sim \mathcal{N}(0, \Lambda^{(h)})} [\sigma(u)\sigma(v)] + 1,\end{aligned}\tag{2.4}$$

for $h = 1, 2, \dots, L$.

To introduce the neural tangent kernel (NTK), we also need to define

$$\dot{\Sigma}^{(h)}(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{(u, v) \sim \mathcal{N}(0, \Lambda^{(h)})} [\dot{\sigma}(u)\dot{\sigma}(v)]\tag{2.5}$$

where $\dot{\sigma}$ denotes the derivative of the activation function σ .

Following the derivation of [33,38], the neural tangent kernel can be generally defined at any training time t , as the neural network parameters $\theta(t)$ are changing during model training by gradient descent. This definition takes the form

$$\text{Ker}_t(\mathbf{x}, \mathbf{x}') = \left\langle \frac{\partial f(\mathbf{x}; \theta(t))}{\partial \theta}, \frac{\partial f(\mathbf{x}'; \theta(t))}{\partial \theta} \right\rangle = \sum_{\theta \in \theta} \frac{\partial f(\mathbf{x}; \theta(t))}{\partial \theta} \frac{\partial f(\mathbf{x}'; \theta(t))}{\partial \theta},\tag{2.6}$$

where $\langle \cdot, \cdot \rangle$ denotes an inner product over θ . This kernel converges in probability to a deterministic kernel $\Theta^{(L)}(\mathbf{x}, \mathbf{x}')$ at random initialization as the width of hidden layers goes to infinity [33]. Specifically,

$$\lim_{d_L \rightarrow \infty} \dots \lim_{d_1 \rightarrow \infty} \text{Ker}_0(\mathbf{x}, \mathbf{x}') = \lim_{d_L \rightarrow \infty} \dots \lim_{d_1 \rightarrow \infty} \left\langle \frac{\partial f(\mathbf{x}; \theta(0))}{\partial \theta}, \frac{\partial f(\mathbf{x}'; \theta(0))}{\partial \theta} \right\rangle = \Theta^{(L)}(\mathbf{x}, \mathbf{x}').\tag{2.7}$$

Here $\Theta^{(L)}(\mathbf{x}, \mathbf{x}')$ denotes the limiting kernel function of a L -layer fully-connected neural network, which is defined by

$$\Theta^{(L)}(\mathbf{x}, \mathbf{x}') = \sum_{h=1}^{L+1} \left(\Sigma^{(h-1)}(\mathbf{x}, \mathbf{x}') \cdot \prod_{h'=h}^{L+1} \dot{\Sigma}^{(h')}(\mathbf{x}, \mathbf{x}') \right)\tag{2.8}$$

where $\dot{\Sigma}^{(L+1)}(\mathbf{x}, \mathbf{x}') = 1$ for convenience. Moreover, Jacot et al. [33] proved that, under some suitable conditions and training time T fixed, Ker_t converges to $\Theta^{(L)}$ for all $0 \leq t \leq T$, as the width goes to infinity. As a consequence, a properly randomly initialized and sufficiently wide deep neural network trained by gradient descent is equivalent to a kernel regression with a deterministic kernel.

3. Physics-informed neural networks (PINNs)

In this section, we study physics-informed neural networks (PINNs) and their corresponding neural tangent kernels. To this end, we consider the following well-posed partial differential equation (PDE) defined on a bounded domain $\Omega \subset \mathbb{R}^d$

$$\mathcal{N}[u](\mathbf{x}) = f(\mathbf{x}), \quad \mathbf{x} \in \Omega\tag{3.1}$$

$$u(\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega\tag{3.2}$$

where $\mathcal{N}$ denotes differential operator and $u(\mathbf{x}) : \bar{\Omega} \rightarrow \mathbb{R}$ is the unknown solution with $\mathbf{x} = (x_1, x_2, \dots, x_d)$. Here we remark that for time-dependent problems, we consider time t as an additional coordinate in $\mathbf{x}$ and Ω denotes the spatio-temporal domain. Then, the initial condition can be simply treated as a special type of Dirichlet boundary condition and included in equation (3.2).

Following the original work of Raissi et al. [39], we assume that the latent solution $u(\mathbf{x})$ can be approximated by a deep neural network $u(\mathbf{x}, \theta)$ with parameters θ , where θ is a collection of all the parameters in the network. We can then define the PDE residual $r(\mathbf{x}; \theta)$ as

$$r(\mathbf{x}; \boldsymbol{\theta}) := \mathcal{N}[u](\mathbf{x}; \boldsymbol{\theta}) - f(\mathbf{x}). \quad (3.3)$$

Note that the parameters of $u(\mathbf{x}; \boldsymbol{\theta})$ can be “learned” by minimizing the following composite loss function

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_b(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}), \quad (3.4)$$

where

$$\mathcal{L}_b(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{N_b} |u(\mathbf{x}_b^i; \boldsymbol{\theta}) - g(\mathbf{x}_b^i)|^2 \quad (3.5)$$

$$\mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{N_r} |r(\mathbf{x}_r^i; \boldsymbol{\theta})|^2. \quad (3.6)$$

Here N_b and N_r denote the batch sizes for the training data $\{\mathbf{x}_b^i, g(\mathbf{x}_b^i)\}_{i=1}^{N_b}$ and $\{\mathbf{x}_r^i, f(\mathbf{x}_r^i)\}_{i=1}^{N_r}$ respectively, which can be randomly sampled at each iteration of a gradient descent algorithm.

3.1. Neural tangent kernel theory for PINNs

In this section we derive the neural tangent kernel of a physics-informed neural network. To this end, consider minimizing the loss function (3.4) by gradient descent with an infinitesimally small learning rate, yielding the continuous-time gradient flow system

$$\frac{d\boldsymbol{\theta}}{dt} = -\nabla \mathcal{L}(\boldsymbol{\theta}), \quad (3.7)$$

and let $u(t) = u(\mathbf{x}_b; \boldsymbol{\theta}(t)) = \{u(\mathbf{x}_b^i; \boldsymbol{\theta}(t))\}_{i=1}^{N_b}$ and $\mathcal{N}[u](t) = \mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t)) = \{\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t))\}_{i=1}^{N_r}$. Then the following lemma characterizes how $u(t)$ and $\mathcal{N}[u](t)$ evolve during training by gradient descent.

Lemma 3.1. Given the data points $\{\mathbf{x}_b^i, g(\mathbf{x}_b^i)\}_{i=1}^{N_b}$, $\{\mathbf{x}_r^i, f(\mathbf{x}_r^i)\}_{i=1}^{N_r}$ and the gradient flow (3.7), $u(t)$ and $\mathcal{N}[u](t)$ obey the following evolution

$$\begin{bmatrix} \frac{du(\mathbf{x}_b; \boldsymbol{\theta}(t))}{dt} \\ \frac{d\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t))}{dt} \end{bmatrix} = - \begin{bmatrix} \mathbf{K}_{uu}(t) & \mathbf{K}_{ur}(t) \\ \mathbf{K}_{ru}(t) & \mathbf{K}_{rr}(t) \end{bmatrix} \cdot \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b) \\ \mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r) \end{bmatrix}, \quad (3.8)$$

where $\mathbf{K}_{ru}(t) = \mathbf{K}_{ur}^T(t)$ and $\mathbf{K}_{uu}(t) \in \mathbb{R}^{N_b \times N_b}$, $\mathbf{K}_{ur}(t) \in \mathbb{R}^{N_b \times N_r}$, and $\mathbf{K}_{rr}(t) \in \mathbb{R}^{N_r \times N_r}$ whose (i, j) -th entry is given by

$$\begin{aligned} (\mathbf{K}_{uu})_{ij}(t) &= \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ (\mathbf{K}_{ur})_{ij}(t) &= \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ (\mathbf{K}_{rr})_{ij}(t) &= \left\langle \frac{d\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \end{aligned} \quad (3.9)$$

Proof. The proof of Lemma 3.1 is given in Appendix A. $\square$

Remark 3.2. $\langle \cdot, \cdot \rangle$ here denotes the inner product over all neural network parameters in $\boldsymbol{\theta}$. For example,

$$(\mathbf{K}_{uu})_{ij}(t) = \sum_{\boldsymbol{\theta} \in \boldsymbol{\theta}} \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \cdot \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}.$$

Remark 3.3. We will denote the matrix $\begin{bmatrix} \mathbf{K}_{uu}(t) & \mathbf{K}_{ur}(t) \\ \mathbf{K}_{ru}(t) & \mathbf{K}_{rr}(t) \end{bmatrix}$ by $\mathbf{K}(t)$ in the following sections. It is easy to see that $\mathbf{K}_{uu}(t)$, $\mathbf{K}_{rr}(t)$ and $\mathbf{K}(t)$ are all positive semi-definite matrices. Indeed, let $\mathbf{J}_u(t)$ and $\mathbf{J}_r(t)$ be the Jacobian matrices of $u(t)$ and $\mathcal{N}[u](t)$ with respect to $\boldsymbol{\theta}$ respectively. Then, we can observe that

$$\mathbf{K}_{uu}(t) = \mathbf{J}_u(t) \mathbf{J}_u^T(t), \quad \mathbf{K}_{rr}(t) = \mathbf{J}_r(t) \mathbf{J}_r^T(t), \quad \mathbf{K}(t) = \begin{bmatrix} \mathbf{J}_u(t) \\ \mathbf{J}_r(t) \end{bmatrix} [\mathbf{J}_u^T(t), \mathbf{J}_r^T(t)].$$

Remark 3.4. It is worth pointing out that equation (3.8) holds for any differential operator $\mathcal{L}$ and any neural network architecture.

The statement of Lemma 3.1 involves the matrix $\mathbf{K}(t)$, which we call the *neural tangent kernel of a physics-informed neural network (NTK of PINNs)*. Recall that an infinitely wide neural network is a Gaussian process, and its NTK remains constant during training [33]. Now two natural questions arise: how does the PDE residual behave in the infinite width limit? Does the NTK of PINNs exhibit similar behavior as the standard NTK? If so, what is the expression of the corresponding kernel? In the next subsections, we will answer these questions and show that, in the infinite-width limit, the NTK of PINNs indeed converges to a deterministic kernel at initialization and then remains constant during training.

4. Analyzing the training dynamics of PINNs through the lens of their NTK

To simplify the proof and understand the key ideas clearly, we confine ourselves to a simple model problem using a fully-connected neural network with one hidden layer. To this end, we consider a one-dimensional Poisson equation as our model problem. Let Ω be a bounded open interval in $\mathbb{R}$. The partial differential equation is summarized as follows

$$\begin{aligned} u_{xx}(x) &= f(x), \quad \forall x \in \Omega \\ u(x) &= g(x), \quad x \in \partial\Omega. \end{aligned} \quad (4.1)$$

We proceed by approximating the solution $u(x)$ by a fully-connected neural network denoted by $u(x; \theta)$ with one hidden layer. Now we define the network explicitly:

$$u(x; \theta) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \sigma(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)}, \quad (4.2)$$

where $\mathbf{W}^{(0)} \in \mathbb{R}^{N \times 1}$, $\mathbf{W}^{(1)} \in \mathbb{R}^{1 \times N}$ are weights, $\mathbf{b}^{(0)} \in \mathbb{R}^N$, $\mathbf{b}^{(1)} \in \mathbb{R}^1$ are biases $\theta = (\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \mathbf{b}^{(0)}, \mathbf{b}^{(1)})$ represents all parameters in the network, and σ is a smooth activation function. Then it is straightforward to show that

$$u_{xx}(x; \theta) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \left[\ddot{\sigma}(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right], \quad (4.3)$$

where $\odot$ denotes point-wise multiplication and $\ddot{\sigma}$ denotes the second order derivative of the activation function σ .

We initialize all the weights and biases to be i.i.d. $\mathcal{N}(0, 1)$ random variables. Based on our presentation in section 2, we already know that, in the infinite width limit, $u(x; \theta)$ is a centered Gaussian process with covariance function $\Sigma^{(1)}(x, x')$ at initialization, which is defined in equation (2.4). The following theorem reveals that $u_{xx}(x; \theta)$ converges in distribution to another centered Gaussian process with a covariance function $\Sigma_{xx}^{(1)}$ under the same limit.

Theorem 4.1. Assume that the activation function σ is smooth and has a bounded second order derivative $\ddot{\sigma}$. Then for a fully-connected neural network of one hidden layer at initialization,

$$u(x; \theta) \xrightarrow{\mathcal{D}} \mathcal{GP}(0, \Sigma^{(1)}(x, x')) \quad (4.4)$$

$$u_{xx}(x; \theta) \xrightarrow{\mathcal{D}} \mathcal{GP}(0, \Sigma_{xx}^{(1)}(x, x')), \quad (4.5)$$

as $N \rightarrow \infty$, where $\mathcal{D}$ means convergence in distribution and

$$\Sigma_{xx}^{(1)}(x, x') = \mathbb{E}_{u, v \sim \mathcal{N}(0, 1)} \left[u^4 \ddot{\sigma}(ux + v) \ddot{\sigma}(ux' + v) \right]. \quad (4.6)$$

Proof. The proof can be found in Appendix B. $\square$

Remark 4.2. By induction, the proof of Theorem 4.1 can be extended to differential operators of any order and fully-connected neural networks with multiple hidden layers. Observe that a linear combination of Gaussian processes is still a Gaussian process. Therefore, Theorem 4.1 can be generalized to any linear partial differential operator under appropriate regularity conditions.

As an immediate corollary, a sufficiently wide physics-informed neural network for model problem (4.1) induces a joint Gaussian process (GP) between the function values and the PDE residual at initialization, indicating a PINNs-GP correspondence for linear PDEs.

The next question we investigate is whether the NTK of PINNs behaves similarly as the NTK of standard neural networks. The next theorem proves that indeed the kernel $\mathbf{K}(0)$ converges in probability to a certain deterministic kernel matrix as the width of the network goes to infinity.

Theorem 4.3. For a physics-informed network with one hidden layer at initialization, and in the limit as the layer's width $N \rightarrow \infty$, the NTK $\mathbf{K}(t)$ of the PINNs model defined in equation (3.9) converges in probability to a deterministic limiting kernel, i.e.,

$$\mathbf{K}(0) = \begin{bmatrix} \mathbf{K}_{uu}(0) & \mathbf{K}_{ur}(0) \\ \mathbf{K}_{ru}(0) & \mathbf{K}_{rr}(0) \end{bmatrix} \rightarrow \begin{bmatrix} \Theta_{uu}^{(1)} & \Theta_{ur}^{(1)} \\ \Theta_{ru}^{(1)} & \Theta_{rr}^{(1)} \end{bmatrix} := \mathbf{K}^*, \quad (4.7)$$

where the explicit expression of $\mathbf{K}^*$ is provided in appendix C.

Proof. The proof can be found in Appendix C. $\square$

Our second key result is that the NTK of PINNs stays asymptotically constant during training, i.e. $\mathbf{K}(t) \approx \mathbf{K}(0)$ for all t . To state and prove the theorem rigorously, we may assume that all parameters and the loss function do not blow up and are uniformly bounded during training. The first two assumptions are both reasonable and practical, otherwise one would obtain unstable and divergent training processes. In addition, the activation has to be 4-th order smooth and all its derivatives are bounded. The last assumption is not a strong restriction since it is satisfied by most of the activation functions commonly used for PINNs such as sigmoids, hyperbolic tangents, sine functions, etc.

Theorem 4.4. For the model problem (4.1) with a fully-connected neural network of one hidden layer, consider minimizing the loss function (3.4) by gradient descent with an infinitesimally small learning rate. For any $T > 0$ satisfying the following assumptions:

- (i) there exists a constant $C > 0$ such that all parameters of the network is uniformly bounded for $t \in T$, i.e.

$$\sup_{t \in [0, T]} \|\boldsymbol{\theta}(t)\|_\infty \leq C$$

where C does not depend on N .

- (ii) there exists a constant $C > 0$ such that

$$\int_0^T \left| \sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \right| d\tau \leq C$$

$$\int_0^T \left| \sum_{i=1}^{N_r} (u_{xx}(x_r^i; \boldsymbol{\theta}(\tau)) - f(x_r^i)) \right| d\tau \leq C$$

- (iii) the activation function σ is smooth and $|\sigma^{(k)}| \leq C$ for $k = 0, 1, 2, 3, 4$, where $\sigma^{(k)}$ denotes k -th order derivative of σ .

Then we have

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \|\mathbf{K}(t) - \mathbf{K}(0)\|_2 = 0, \quad (4.8)$$

where $\mathbf{K}(t)$ is the corresponding NTK of PINNs.

Proof. The proof can be found in Appendix D. $\square$

Here we provide some intuition behind the proof. The crucial observation is that all parameters of the network change little during training (see Lemma D.2 in the Appendix). By intuition, for sufficient wide neural networks, any slight movement of weights would contribute to a non-negligible change in the network output. As a result, the gradients of the outputs $u(x, \boldsymbol{\theta})$ and $u_{xx}(x, \boldsymbol{\theta})$ with respect to parameters barely change (see Lemma D.4), and, therefore, the kernel remains almost static during training.

Combining Theorem 4.3 and Theorem 4.4 we may conclude that, for the model problem of equation (4.1), we have

$$\mathbf{K}(t) \approx \mathbf{K}(0) \approx \mathbf{K}^*, \quad \forall t,$$

from which (and equation (3.8)) we immediately obtain

$$\begin{bmatrix} \frac{du(\mathbf{x}_b; \boldsymbol{\theta}(t))}{dt} \\ \frac{du_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t))}{dt} \end{bmatrix} = -\mathbf{K}(t) \cdot \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r) \end{bmatrix} \approx -\mathbf{K}^* \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r) \end{bmatrix}. \quad (4.9)$$

Note that if the matrix $\mathbf{K}^*$ is invertible, then according to [40,31], the network's outputs $u(x, \boldsymbol{\theta})$ and $u_{xx}(x, \boldsymbol{\theta})$ can be approximated for any arbitrary test data $\mathbf{x}_{\text{test}}$ after t steps of gradient descent as

$$\begin{bmatrix} u(\mathbf{x}_{test}; \boldsymbol{\theta}(t)) \\ u_{xx}(\mathbf{x}_{test}; \boldsymbol{\theta}(t)) \end{bmatrix} \approx \mathbf{K}_{test}^* (\mathbf{K}^*)^{-1} (I - e^{-\mathbf{K}^* t}) \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix}, \quad (4.10)$$

where $\mathbf{K}_{test}$ is the NTK matrix between all points in $\mathbf{x}_{test}$ and all training data. Letting $t \rightarrow \infty$, we obtain

$$\begin{bmatrix} u(\mathbf{x}_{test}; \boldsymbol{\theta}(\infty)) \\ u_{xx}(\mathbf{x}_{test}; \boldsymbol{\theta}(\infty)) \end{bmatrix} \approx \mathbf{K}_{test}^* (\mathbf{K}^*)^{-1} \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix}.$$

This implies that, under the assumption that $\mathbf{K}^*$ is invertible, an infinitely wide physics-informed neural network for model problem (4.1) is also equivalent to a kernel regression. However, from the authors' experience, the NTK of PINNs is always degenerate (see Figs. 2c, 3c) which means that we cannot invert the kernel matrix and thus be able to casually perform kernel regression predictions in practice.

5. Spectral bias in physics-informed neural networks

In this section, we will utilize the developed theory to investigate whether physics-informed neural networks are spectrally biased. The term ‘‘spectral bias’’ [29,41,32] refers to a well known pathology that prevents deep fully-connected networks from learning high-frequency functions.

Since the NTK of PINNs barely changes during training, we may rewrite equation (4.9) as

$$\begin{bmatrix} \frac{du(\mathbf{x}_b; \boldsymbol{\theta}(t))}{dt} \\ \frac{du_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t))}{dt} \end{bmatrix} \approx -\mathbf{K}(0) \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r) \end{bmatrix}, \quad (5.1)$$

which leads to

$$\begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) \end{bmatrix} \approx (I - e^{-\mathbf{K}(0)t}) \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix}. \quad (5.2)$$

As mentioned in Remark 3.3, the NTK of PINNs is also positive semi-definite. So we can take its spectral decomposition $\mathbf{K}(0) = \mathbf{Q}^T \boldsymbol{\Lambda} \mathbf{Q}$, where $\mathbf{Q}$ is an orthogonal matrix and $\boldsymbol{\Lambda}$ is a diagonal matrix whose entries are the eigenvalues $\lambda_i \geq 0$ of $\mathbf{K}(0)$. Consequently, the training error is given by

$$\begin{aligned} \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) \end{bmatrix} - \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix} &\approx (I - e^{-\mathbf{K}(0)t}) \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix} - \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix} \\ &\approx -\mathbf{Q}^T e^{-\boldsymbol{\Lambda} t} \mathbf{Q} \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix}, \end{aligned}$$

which is equivalent to

$$\mathbf{Q} \left(\begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) \\ u_{xx}(\mathbf{x}_r; \boldsymbol{\theta}(t)) \end{bmatrix} - \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix} \right) \approx -e^{-\boldsymbol{\Lambda} t} \mathbf{Q} \cdot \begin{bmatrix} g(\mathbf{x}_b) \\ f(\mathbf{x}_r) \end{bmatrix}. \quad (5.3)$$

This implies that the i -th component of the left hand side in equation (5.3) will decay approximately at the rate $e^{-\lambda_i t}$. In other words, the eigenvalues of the kernel characterize how fast the absolute training error decreases. Particularly, components of the target function that correspond to kernel eigenvectors with larger eigenvalues will be learned faster. For fully-connected networks, the eigenvectors corresponding to higher eigenvalues of the NTK matrix generally exhibit lower frequencies [29,42,32]. From Fig. 1, one can observe that the eigenvalues of the NTK of PINNs decay rapidly. This results in extremely slow convergence to the high-frequency components of the target function. Thus we may conclude that PINNs suffer from the spectral bias either.

More generally, the NTK of PINNs after t steps of gradient descent is given by

$$\mathbf{K}(t) = \begin{bmatrix} \mathbf{K}_{uu}(t) & \mathbf{K}_{ur}(t) \\ \mathbf{K}_{ru}(t) & \mathbf{K}_{rr}(t) \end{bmatrix} = \begin{bmatrix} \mathbf{J}_u(t) \\ \mathbf{J}_r(t) \end{bmatrix} [\mathbf{J}_u^T(t), \mathbf{J}_r^T(t)] = \mathbf{J}(t) \mathbf{J}^T(t).$$

It follows that

$$\begin{aligned} \sum_{i=1}^{N_b+N_r} \lambda_i(t) &= \text{Tr}(\mathbf{K}(t)) = \text{Tr}(\mathbf{J}(t) \mathbf{J}^T(t)) = \text{Tr}(\mathbf{J}^T(t) \mathbf{J}(t)) \\ &= \text{Tr}(\mathbf{J}_u^T(t) \mathbf{J}_u(t) + \mathbf{J}_r^T(t) \mathbf{J}_r(t)) = \text{Tr}(\mathbf{J}_u(t) \mathbf{J}_u^T(t)) + \text{Tr}(\mathbf{J}_r(t) \mathbf{J}_r^T(t)) \\ &= \sum_{i=1}^{N_b} \lambda_i^{uu}(t) + \sum_{i=1}^{N_r} \lambda_i^{rr}(t), \end{aligned}$$

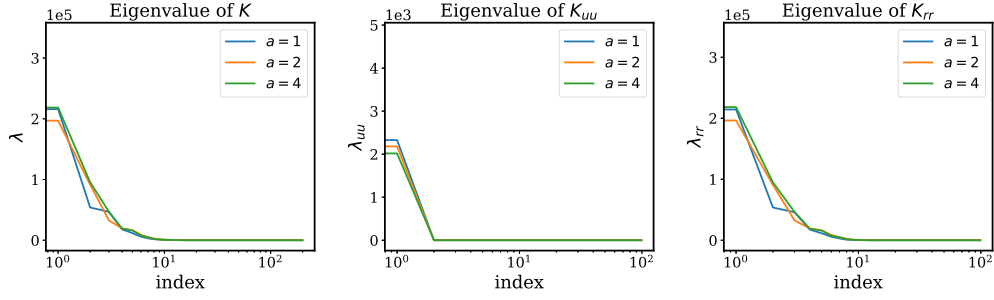


Fig. 1. Model problem (1D Poisson equation): The eigenvalues of $\mathbf{K}$, $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$ at initialization in descending order for different fabricated solutions $u(x) = \sin(a\pi x)$ where $a = 1, 2, 4$.

where $\lambda_i(t)$, $\lambda_i^{uu}(t)$ and $\lambda_i^{rr}(t)$ denote the eigenvalues of $\mathbf{K}(t)$, $\mathbf{K}_{uu}(t)$ and $\mathbf{K}_{rr}(t)$, respectively. This reveals that the overall convergence rate of the total training error is characterized by the eigenvalues of $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$ together. Meanwhile, the separate training error of $u(\mathbf{x}_b; \theta)$ and $u_{xx}(\mathbf{x}_r; \theta)$ is determined by the eigenvalues of $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$, respectively. The above observation motivates us to give the following definition.

Definition 5.1. For a positive semi-definite kernel matrix $\mathbf{K} \in \mathbb{R}^{n \times n}$, the average convergence rate c is defined as the mean of all its eigenvalues λ_i 's, i.e.

$$c = \frac{\sum_{i=1}^n \lambda_i}{n} = \frac{\text{Tr}(\mathbf{K})}{n}. \quad (5.4)$$

In particular, for any two kernel matrices $\mathbf{K}_1, \mathbf{K}_2$ with average convergence rate c_1 and c_2 respectively, we say that $\mathbf{K}_1$ dominates $\mathbf{K}_2$ if $c_1 \gg c_2$.

As a concrete example, we train a fully-connected neural network with one hidden layer and 100 neurons to solve the model Problem 7.1 with a fabricated solution $u(x) = \sin(a\pi x)$ for different frequency amplitudes a . Fig. 1 shows all eigenvalues of $\mathbf{K}$, $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$ at initialization in descending order. As with conventional deep fully-connected networks, the eigenvalues of the PINNs' NTK decay rapidly and most of the eigenvalues are near zero. Moreover, the distribution of eigenvalues of $\mathbf{K}$ looks similar for different frequency functions (different a), which may heuristically explain that PINNs tend to learn all frequencies almost simultaneously, as observed in Lu et al. [43].

Another key observation here is that the eigenvalues of $\mathbf{K}_{rr}$ are much greater than $\mathbf{K}_{uu}$, namely $\mathbf{K}_{rr}$ dominates $\mathbf{K}_{uu}$ by Definition 5.1. As a consequence, the PDE residual converges much faster than fitting the PDE boundary conditions, which may prevent the network from approximating the correct solution. From the authors' experience, high frequency functions typically lead to high eigenvalues in $\mathbf{K}_{rr}$, but in some cases $\mathbf{K}_{uu}$ can dominate $\mathbf{K}_{rr}$. We believe that such a discrepancy between $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$ is one of the key fundamental reasons behind why PINNs can often fail to train and yield accurate predictions. In light of this evidence, in the next section, we describe a practical technique to address this pathology by appropriately assigning weights to the different terms in a PINNs loss function.

6. Practical insights

In this section, we consider general PDEs of the form (3.1) - (3.2) by leveraging the NTK theory we developed for PINNs. We approximate the latent solution $u(\mathbf{x})$ by a fully-connected neural network $u(\mathbf{x}, \theta)$ with multiple hidden layers, and train its parameters θ by minimizing the following composite loss function

$$\mathcal{L}(\theta) = \mathcal{L}_b(\theta) + \mathcal{L}_r(\theta) \quad (6.1)$$

$$= \frac{\lambda_b}{2N_b} \sum_{i=1}^{N_b} |u(\mathbf{x}_b^i; \theta) - g(\mathbf{x}_b^i)|^2 + \frac{\lambda_r}{2N_r} \sum_{i=1}^{N_r} |r(\mathbf{x}_r^i; \theta)|^2, \quad (6.2)$$

where λ_b and λ_r are some hyper-parameters which may be tuned manually or automatically by utilizing the back-propagated gradient statistics during training [28]. Here, the training data $\{\mathbf{x}_b^i, g(\mathbf{x}_b^i)\}_{i=1}^{N_b}$ and $\{\mathbf{x}_r^i, f(\mathbf{x}_b^i)\}$ may correspond to the full data-batch or mini-batches that are randomly sampled at each iteration of gradient descent.

Similar to the proof of Lemma 3.1, we can derive the dynamics of the outputs $u(\mathbf{x}, \theta)$ and $u_{xx}(\mathbf{x}, \theta)$ corresponding to the above loss function as

$$\begin{bmatrix} \frac{du(\mathbf{x}_b; \theta(t))}{dt} \\ \frac{d\mathcal{N}[u](\mathbf{x}_r; \theta(t))}{dt} \end{bmatrix} = - \begin{bmatrix} \frac{\lambda_b}{N_b} \mathbf{K}_{uu}(t) & \frac{\lambda_r}{N_r} \mathbf{K}_{ur}(t) \\ \frac{\lambda_b}{N_b} \mathbf{K}_{ru}(t) & \frac{\lambda_r}{N_r} \mathbf{K}_{rr}(t) \end{bmatrix} \cdot \begin{bmatrix} u(\mathbf{x}_b; \theta(t)) - g(\mathbf{x}_b) \\ \mathcal{N}[u](\mathbf{x}_r; \theta(t)) - f(\mathbf{x}_r) \end{bmatrix} \quad (6.3)$$

$$:= -\tilde{\mathbf{K}}(t) \cdot \begin{bmatrix} u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b) \\ \mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r) \end{bmatrix}, \quad (6.4)$$

where $\mathbf{K}_{uu}$, $\mathbf{K}_{ur}$ and $\mathbf{K}_{rr}$ are defined to be the same as in equation (3.9). From simple stability analysis of a gradient descent (i.e. forward Euler [44]) discretization of above ODE system, the maximum learning rate should be less than or equal to $2/\lambda_{\max}(\tilde{\mathbf{K}}(t))$. Also note that an alternative mechanism for controlling stability is to increase the batch size, which effectively corresponds to decreasing the learning rate. Recall that the current setup in the main theorems put forth in this work holds for the model problem in equation (4.1) and fully-connected networks of one hidden layer with an NTK parameterization. This implies that, for general nonlinear PDEs, the NTK of PINNs may not remain fixed during training. Nevertheless, as mentioned in Remark 3.4, we emphasize that, given an infinitesimal learning rate, equation (6.3) holds for any network architecture and any differential operator. Similarly, the singular values of NTK $\tilde{\mathbf{K}}(t)$ determine the convergence rate of the training error using singular value decomposition, since $\tilde{\mathbf{K}}(t)$ may not necessarily be semi-positive definite. Therefore, we can still understand the training dynamics of PINNs by tracking their NTK $\tilde{\mathbf{K}}(t)$ during training, even for general nonlinear PDE problems.

A key observation here is that the magnitude of λ_b , λ_r , as well as the size of mini-batch would have a crucial impact on the singular values of $\tilde{\mathbf{K}}(t)$, and, thus, the convergence rate of the training error of $u(\mathbf{x}_b; \boldsymbol{\theta})$ and $\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta})$. For instance, if we increase λ_b and fix the batch size N_b , N_r and the weight λ_r , then this will improve the convergence rate of $u(\mathbf{x}_b; \boldsymbol{\theta})$. Furthermore, in the sense of convergence rate, changing the weights λ_b or λ_r is equivalent to changing the corresponding batch size N_b , N_r . Based on these observations, we can overcome the discrepancy between $\mathbf{K}_{uu}$ and $\mathbf{K}_{rr}$ discussed in section 5 by calibrating the weights or batch size such that each component of $u(\mathbf{x}_r; \boldsymbol{\theta})$ and $\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta})$ has similar convergence rate in magnitude. Since manipulating the batch size may involve extra computational costs (e.g., it may result to prohibitively very large batches), here we fix the batch size and just consider adjusting the weights λ_b or λ_r according to Algorithm 1.

Algorithm 1: Adaptive weights for physics-informed neural networks.

Consider a physics-informed neural network $u(\mathbf{x}; \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ and a loss function

$$\mathcal{L}(\boldsymbol{\theta}) := \lambda_b \mathcal{L}_b(\boldsymbol{\theta}) + \lambda_r \mathcal{L}_r(\boldsymbol{\theta}),$$

where $\mathcal{L}_r(\boldsymbol{\theta})$ denotes the PDE residual loss and $\mathcal{L}_b(\boldsymbol{\theta})$ corresponds to boundary conditions. Initialize the weights λ_b, λ_r to 1 and use S steps of a gradient descent algorithm to update the parameters $\boldsymbol{\theta}$ as:

for $n = 1, \dots, S$ **do**

(a) Compute λ_b and λ_r by

$$\lambda_b = \frac{\sum_{i=1}^{N_r+N_b} \lambda_i(n)}{\sum_{i=1}^{N_b} \lambda_i^{uu}(n)} = \frac{\text{Tr}(\mathbf{K}(n))}{\text{Tr}(\mathbf{K}_{uu}(n))} \quad (6.5)$$

$$\lambda_r = \frac{\sum_{i=1}^{N_r+N_b} \lambda_i(n)}{\sum_{i=1}^{N_r} \lambda_i^{rr}(n)} = \frac{\text{Tr}(\mathbf{K}(n))}{\text{Tr}(\mathbf{K}_{rr}(n))} \quad (6.6)$$

where $\lambda_i(n)$, $\lambda_i^{uu}(n)$ and $\lambda_i^{rr}(n)$ are eigenvalues of $\mathbf{K}(n)$, $\mathbf{K}_{uu}(n)$, $\mathbf{K}_{rr}(n)$ at n -th iteration.

(b) Update the parameters $\boldsymbol{\theta}$ via gradient descent

$$\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_n) \quad (6.7)$$

end

First we remark that the updates in equations (6.5) and (6.6) can either take place at every iteration of the gradient descent loop, or at a frequency specified by the user (e.g., every 10 gradient descent steps). To compute the sum of eigenvalues, it suffices to compute the trace of the corresponding NTK matrices, which can save some computational resources. Besides, we point out that the computation of the NTK $\mathbf{K}(t)$ is associated with the training data points fed to the network at each iteration, which means that the values of the kernel are not necessarily same at each iteration. However, if we assume that all training data points are sampled from the same distribution and the change of NTK at each iteration is negligible, then the computed kernel should be approximately equal up to a permutation matrix. As a result, the change of eigenvalues of $\mathbf{K}(t)$ at each iteration is also negligible and thus the training process of Algorithm 1 should be stable. In section 7.2, we performed detailed numerical experiments to validate the effectiveness of the proposed algorithm.

Here we also note that, in previous work, Wang et al. introduced an alternative empirical approach for automatically tuning the weights λ_b or λ_r with the goal of balancing the magnitudes of the back-propagated gradients originating from different terms in a PINNs loss function. While effective in practice, this approach lacked any theoretical justification and did not provide a deeper insight into the training dynamics of PINNs. In contrast, the approach presented here follows naturally from the NTK theory derived in section 4, and aims to trace and tackle the pathological convergence behavior of PINNs at its root.

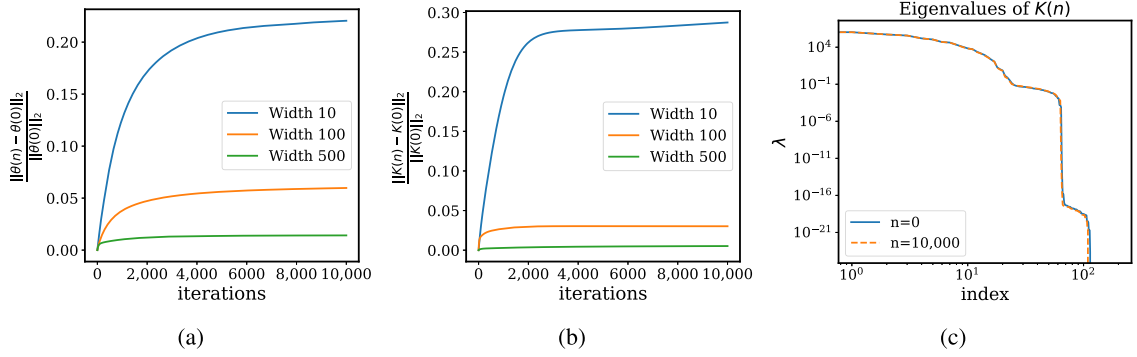


Fig. 2. Model Problem 7.1 (1D Poisson equation): (a) (b) The relative change of parameters θ and the NTK of PINNs K obtained by training a fully-connected neural network with one hidden layer and different widths (10, 100, 500) via 10,000 iterations of full-batch gradient descent with a learning rate of 10^{-5} . (c) The eigenvalues of the NTK K at initialization and at the last step ($n = 10^4$) of training a width = 500 fully-connected neural network.

7. Numerical experiments

In this section, we provide a series of numerical studies that aim to validate our theory or access the performance of the proposed algorithm against the standard PINNs [27] for inferring the solution of PDEs. Throughout numerical experiments we will approximate the latent variables by fully-connected neural networks with NTK parameterization (2.3) and hyperbolic tangent activation functions. All networks are trained using standard stochastic gradient descent, unless otherwise specified. Finally, all results presented in this section can be reproduced using our publicly available code <https://github.com/PredictiveIntelligenceLab/PINNsNTK>.

7.1. Convergence of the NTK of PINNs

As our first numerical example, we still focus on the model problem (4.1) and verify the convergence of the PINNs' NTK. Specifically, we set Ω to be the unit interval $[0, 1]$ and fabricate the exact solution to this problem taking the form $u(x) = \sin(\pi x)$. The corresponding f and g are given by

$$\begin{aligned} f(x) &= -\pi^2 \sin(\pi x), \quad x \in [0, 1] \\ g(x) &= 0, \quad x = 0, 1. \end{aligned}$$

We proceed by approximating the latent solution $u(x)$ by a fully-connected neural network $u(x; \theta)$ of one hidden layer with NTK parameterization (see equation (2.3)), and a hyperbolic tangent activation function. The corresponding loss function is given by

$$\mathcal{L}(\theta) = \mathcal{L}_b(\theta) + \mathcal{L}_r(\theta) \quad (7.1)$$

$$= \frac{1}{2} \sum_{i=1}^{N_b} |u(\mathbf{x}_b^i; \theta) - g(\mathbf{x}_b^i)|^2 + \frac{1}{2} \sum_{i=1}^{N_r} |u_{xx}(\mathbf{x}_r^i; \theta) - f(\mathbf{x}_r^i)|^2 \quad (7.2)$$

Here we choose $N_b = N_r = 100$ and the collocation points $\{\mathbf{x}_r^i\}_{i=1}^{N_r}$ are uniformly spaced in the unit interval. To monitor the change of the NTK $K(t)$ for this PINN model, we train the network for different widths and for 10,000 iterations by minimizing the loss function given above using standard full-batch gradient descent with a learning rate of 10^{-5} . Here we remark that, in order to keep the gradient descent dynamics (3.8) steady, the learning rate should be less than $2/\lambda_{\max}$, where $\lambda_{\max}$ denotes the maximum eigenvalue of $K(t)$.

Fig. 2a and 2b present the relative change in the norm of network's weights and NTK (starting from a random initialization) during training. As it can be seen, the change of both the weights and the NTK tends to zero as the width of the network grows to infinity, which is consistent with Lemma D.2 and Theorem 4.4. Moreover, we know that convergence in a matrix norm implies convergence in eigenvalues, and eigenvalues characterize the properties of a given matrix. To this end, we compute and monitor all eigenvalues of $K(t)$ of the network for width = 500 at initialization and after 10,000 steps of gradient and plot them in descending order in Fig. 2c. As expected, we see that all eigenvalues barely change for these two snapshots. Based on these observations, we may conclude that the NTK of PINNs with one hidden layer stays almost fixed during training.

However, PINNs of multiple hidden layers are not covered by our theory at the moment. Out of interest, we also investigate the relative change of weights, kernel, as well as the kernel's eigenvalues for a fully-connected network with three hidden layers (see Fig. 3). We can observe that the change in both the weights and the NTK behaves almost identical to the

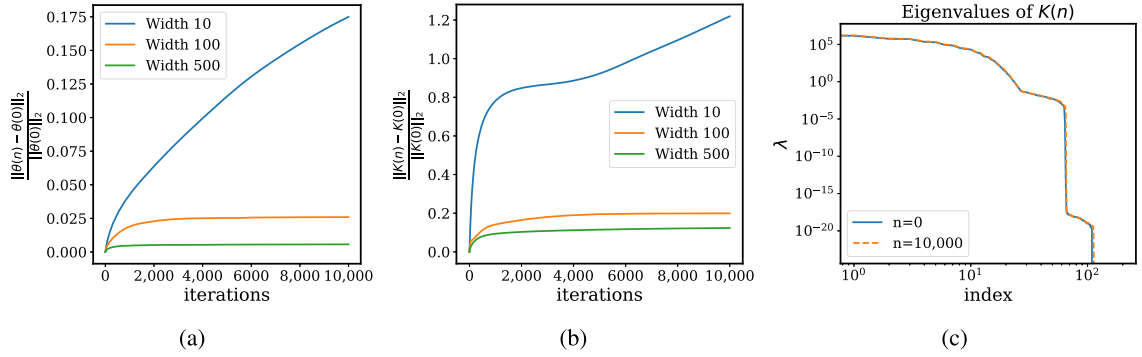


Fig. 3. Model Problem 7.1 (1D Poisson equation): (a) (b) The relative change of parameters θ and the NTK of PINNs K obtained by training a fully-connected neural network with three hidden layers and different widths (10, 100, 500) via 10,000 iterations of full-batch gradient descent with a learning rate of 10^{-5} . (c) The eigenvalues of the NTK K at initialization and at the last step ($n = 10^4$) of training a width = 500 fully-connected neural network.

case of a fully-connected network with one hidden layer shown in Fig. 2. Therefore we may conjecture that, for any linear or even nonlinear PDEs, the NTK of PINNs converges to a deterministic kernel and remains constant during training in the infinite width limit.

7.2. Adaptive training for PINNs

In this section, we aim to validate the developed theory and examine the effectiveness of the proposed adaptive training algorithm on the model problem of equation 7.1. To this end, we consider a fabricated exact solution of the form $u(x) = \sin(4\pi x)$, inducing a corresponding forcing term f and Dirichlet boundary condition g given by

$$f(x) = -16\pi^2 \sin(4\pi x), \quad x \in [0, 1]$$

$$g(x) = 0, \quad x = 0, 1.$$

We proceed by approximating the latent solution $u(x)$ by a fully-connected neural network with one hidden layer and width set to 100. Recall from Theorem 4.3 and Theorem 4.4, that the NTK barely changes during training. This implies that the weights λ_b, λ_r are determined by NTK at initialization and thus they can be regarded as fixed weights during training. Moreover, from Fig. 1, we already know that K_{rr} dominates K_{uu} for this example. Therefore, the updating rule for hyper-parameters λ_b, λ_r at t step of gradient descent can be reduced to

$$\lambda_b = \frac{\sum_{i=1}^{N_b+N_r} \lambda_i(t)}{\sum_{i=1}^{N_b} \lambda_i^{uu}(t)} \approx \frac{\sum_{i=1}^{N_r} \lambda_i^{rr}(t)}{\sum_{i=1}^{N_b} \lambda_i^{uu}(t)} \approx \frac{\text{Tr}(K_{rr}(0))}{\text{Tr}(K_{uu}(0))} \quad (7.3)$$

$$\lambda_r = \frac{\sum_{i=1}^{N_b+N_r} \lambda_i(t)}{\sum_{i=1}^{N_r} \lambda_i^{rr}(t)} \approx 1. \quad (7.4)$$

We proceed by training the network via full-batch gradient descent with a learning rate of 10^{-5} to minimize the following loss function

$$\begin{aligned} \mathcal{L}(\theta) &= \mathcal{L}_b(\theta) + \mathcal{L}_r(\theta) \\ &= \frac{\lambda_b}{2N_b} \sum_{i=1}^{N_b} |u(\mathbf{x}_b^i; \theta) - g(\mathbf{x}_b^i)|^2 + \frac{\lambda_r}{2N_r} \sum_{i=1}^{N_r} |u_{xx}(\mathbf{x}_r^i; \theta) - f(\mathbf{x}_r^i)|^2, \end{aligned}$$

where the batch sizes are $N_b = N_r = 100$, $\lambda_r = 1$ and the computed $\lambda_b \approx 100$.

A comparison of predicted solution $u(x)$ between the original PINNs ($\lambda_b = \lambda_r = 1$) and PINNs with adaptive weights after 40,000 iterations are shown in Fig. 4. It can be observed that the proposed algorithm yields a much more accurate predicted solution and improves the relative L^2 error by about two orders of magnitude. Furthermore, we also investigate how the predicted performance of PINNs depends on the choice of different weights in the loss function. To this end, we fix $\lambda_r = 1$ and train the same network, but now we manually tune λ_b . Fig. 5 presents a visual assessment of relative L^2 errors of predicted solutions for different $\lambda_b \in [1, 500]$ averaged over ten independent trials. One can see that the relative L^2 error decreases rapidly to a local minimum as λ_b increases from 1 to about 100 and then shows oscillations as λ_b continues to

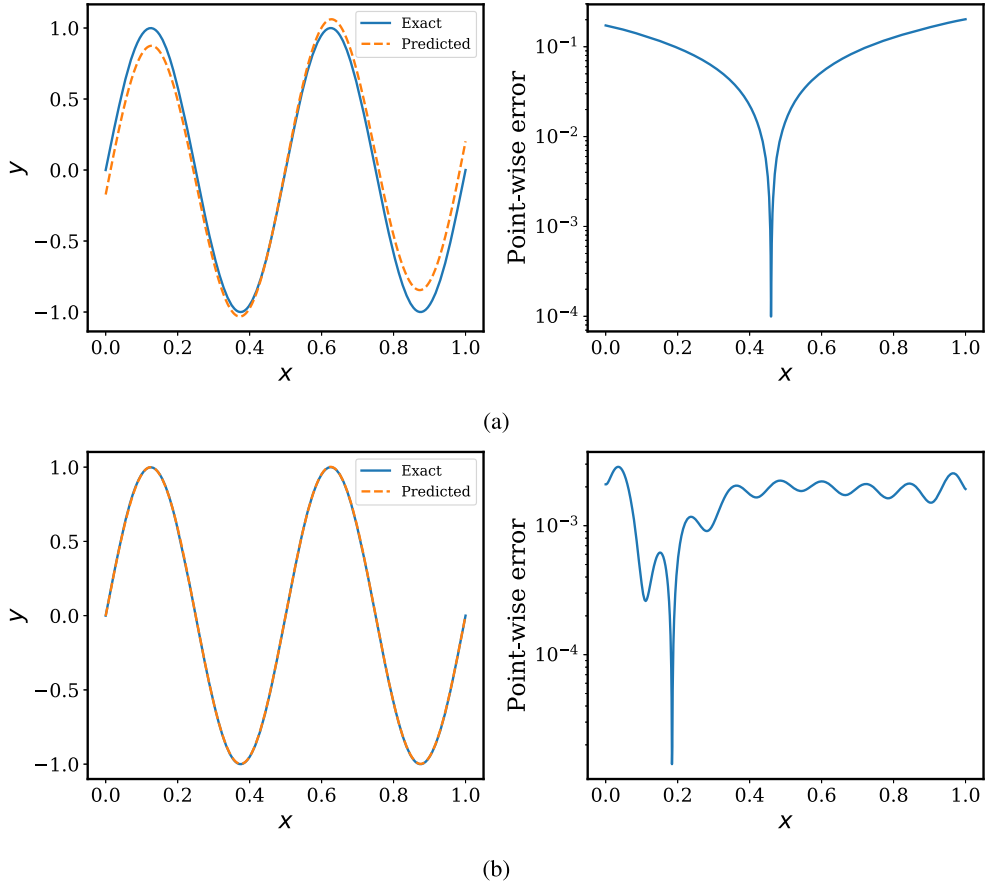


Fig. 4. Model Problem 7.2 (1D Poisson equation): (a) The predicted solution against the exact solution obtained by training a fully-connected neural network of one hidden layer with width = 100 via 40,000 iterations of full-batch gradient descent with a learning rate of 10^{-5} . The relative L^2 error is $2.40e-01$. (b) The predicted solution against the exact solution obtained by training the same neural network using fixed weights $\lambda_b = 100, \lambda_r = 1$ via 40,000 iterations of full-batch gradient descent with a learning rate of 10^{-5} . The relative L^2 error is $1.63e-03$.

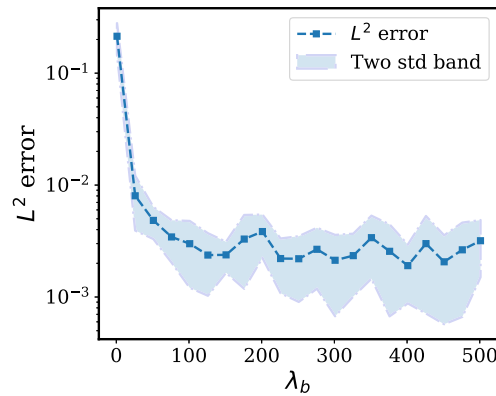


Fig. 5. Model problem of equation 7.2 (1D Poisson equation): The relative L^2 error of predicted solutions averaged over 10 independent trials by training a fully-connected neural network of one hidden layer with width = 100 using different fixed weights $\lambda_b \in [1, 500]$ for 40,000 gradient descent iterations.

increase. Moreover, a large magnitude of λ_b seems to lead to a large standard deviation in the L^2 error, which may be due to the imaginary eigenvalues of the indefinite kernel $\tilde{\mathbf{K}}$ resulting in an unstable training process. This empirical simulation study confirms that the weights $\lambda_r = 1$ and λ_b suggested by our theoretical analysis based on analyzing the NTK spectrum are robust and closely agree with the optimal weights obtained via manual hyper-parameter tuning.

7.3. One-dimensional wave equation

As our last example, we present a study that demonstrates the effectiveness of Algorithm 1 in a practical problem for which conventional PINNs models face severe difficulties. To this end, we consider a one-dimensional wave equation in the domain $\Omega = [0, 1] \times [0, 1]$ taking the form

$$u_{tt}(x, t) - 4u_{xx}(x, t) = 0, \quad (x, t) \in (0, 1) \times (0, 1) \quad (7.5)$$

$$u(0, t) = u(1, t) = 0, \quad t \in [0, 1] \quad (7.6)$$

$$u(x, 0) = \sin(\pi x) + \frac{1}{2} \sin(4\pi x), \quad x \in [0, 1] \quad (7.7)$$

$$u_t(x, 0) = 0, \quad x \in [0, 1]. \quad (7.8)$$

First, by d'Alembert's formula [45], the solution $u(x, t)$ is given by

$$u(x, t) = \sin(\pi x) \cos(2\pi t) + \frac{1}{2} \sin(4\pi x) \cos(8\pi t). \quad (7.9)$$

Here we treat the temporal coordinate t as an additional spatial coordinate in $\mathbf{x}$ and then the initial condition (7.7) can be included in the boundary condition (7.6), namely

$$u(\mathbf{x}) = g(\mathbf{x}), \quad \mathbf{x} \in \partial\Omega$$

Now we approximate the solution u by a 5-layer deep fully-connected network $u(\mathbf{x}, \theta)$ with 500 neurons per hidden layer, where $\mathbf{x} = (x, t)$. Then we can formulate a “physics-informed” loss function by

$$\mathcal{L}(\theta) = \lambda_u \mathcal{L}_u(\theta) + \lambda_{u_t} \mathcal{L}_{u_t}(\theta) + \lambda_r \mathcal{L}_r(\theta) \quad (7.10)$$

$$= \frac{\lambda_u}{2N_u} \sum_{i=1}^{N_u} |u(\mathbf{x}_u^i; \theta) - g(\mathbf{x}_u^i)|^2 + \frac{\lambda_{u_t}}{2N_{u_t}} \sum_{i=1}^{N_{u_t}} |u_t(\mathbf{x}_{u_t}^i; \theta)|^2 + \frac{\lambda_r}{2N_r} \sum_{i=1}^{N_r} |\mathcal{N}u(\mathbf{x}_r^i; \theta)|^2, \quad (7.11)$$

where the hyper-parameters $\lambda_u, \lambda_{u_t}, \lambda_r$ are initialized to 1, the batch sizes are set to $N_u = N_{u_t} = N_r = 300$, and $\mathcal{N} = \partial_{tt} - 4\partial_{xx}$. Here all training data are uniformly sampling inside the computational domain at each gradient descent iteration. The network $u(\mathbf{x}; \theta)$ is initialized using the standard Glorot scheme [46] and then trained by minimizing the above loss function via stochastic gradient descent using the Adam optimizer with default settings [47]. Fig. 6a provides a comparison between the predicted solution against the ground truth obtained after 80,000 training iterations. Clearly the original PINN model fails to approximate the ground truth solution and the relative L^2 error is above 40%.

To explore the reason behind PINN's failure for this example, we compute its NTK and track it during training. Similar to the proof of Lemma 3.1, the corresponding NTK can be derived from the loss function (7.10)

$$\begin{bmatrix} \frac{du(\mathbf{x}_u; \theta(t))}{dt} \\ \frac{du_t(\mathbf{x}_{u_t}; \theta(t))}{dt} \\ \frac{d\mathcal{N}[u](\mathbf{x}_r; \theta(t))}{dt} \end{bmatrix} := \tilde{\mathbf{K}}(t) \cdot \begin{bmatrix} u(\mathbf{x}_b; \theta(t)) - g(\mathbf{x}_b) \\ u_t(\mathbf{x}_{u_t}; \theta(t)) \\ \mathcal{N}[u](\mathbf{x}_r; \theta(t)) \end{bmatrix}, \quad (7.12)$$

where

$$\begin{aligned} \tilde{\mathbf{K}}(t) &= \begin{bmatrix} \frac{\lambda_u}{N_u} \mathbf{J}_u(t) \\ \frac{\lambda_{u_t}}{N_{u_t}} \mathbf{J}_{u_t}(t) \\ \frac{\lambda_r}{N_r} \mathbf{J}_r(t) \end{bmatrix} \cdot [\mathbf{J}_u^T(t), \mathbf{J}_{u_t}^T(t), \mathbf{J}_r^T(t)], \\ [\mathbf{K}_u(t)]_{ij} &= [\mathbf{J}_u(t) \mathbf{J}_u^T(t)]_{ij} = \left\langle \frac{du(\mathbf{x}_u^i; \theta(t))}{d\theta}, \frac{du(\mathbf{x}_u^j; \theta(t))}{d\theta} \right\rangle \\ [\mathbf{K}_{u_t}(t)]_{ij} &= [\mathbf{J}_{u_t}(t) \mathbf{J}_{u_t}^T(t)]_{ij} = \left\langle \frac{du_t(\mathbf{x}_{u_t}^i; \theta(t))}{d\theta}, \frac{du_t(\mathbf{x}_{u_t}^j; \theta(t))}{d\theta} \right\rangle \\ [\mathbf{K}_r(t)]_{ij} &= [\mathbf{J}_r(t) \mathbf{J}_r^T(t)]_{ij} = \left\langle \frac{d\mathcal{N}[u](\mathbf{x}_r^i; \theta(t))}{d\theta}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \theta(t))}{d\theta} \right\rangle. \end{aligned}$$

A visual assessment of the eigenvalues of $\mathbf{K}_u$, $\mathbf{K}_{u_t}$ and $\mathbf{K}_r$ at initialization and the last step of gradient descent are presented in Fig. 7. It can be observed that the NTK does not remain fixed and all eigenvalues move “outward” in the beginning of the training, and then remain almost static such that $\mathbf{K}_r$ and $\mathbf{K}_{u_t}$ dominate $\mathbf{K}_u$ during training. Consequently, the components

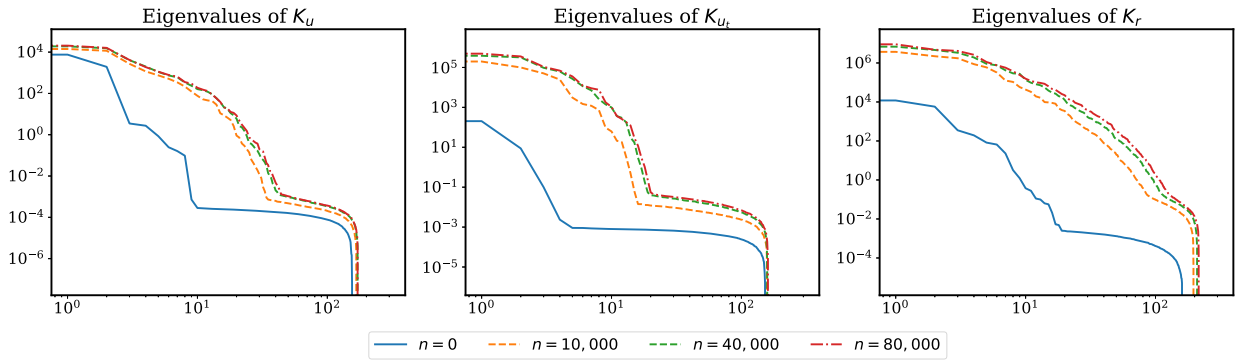
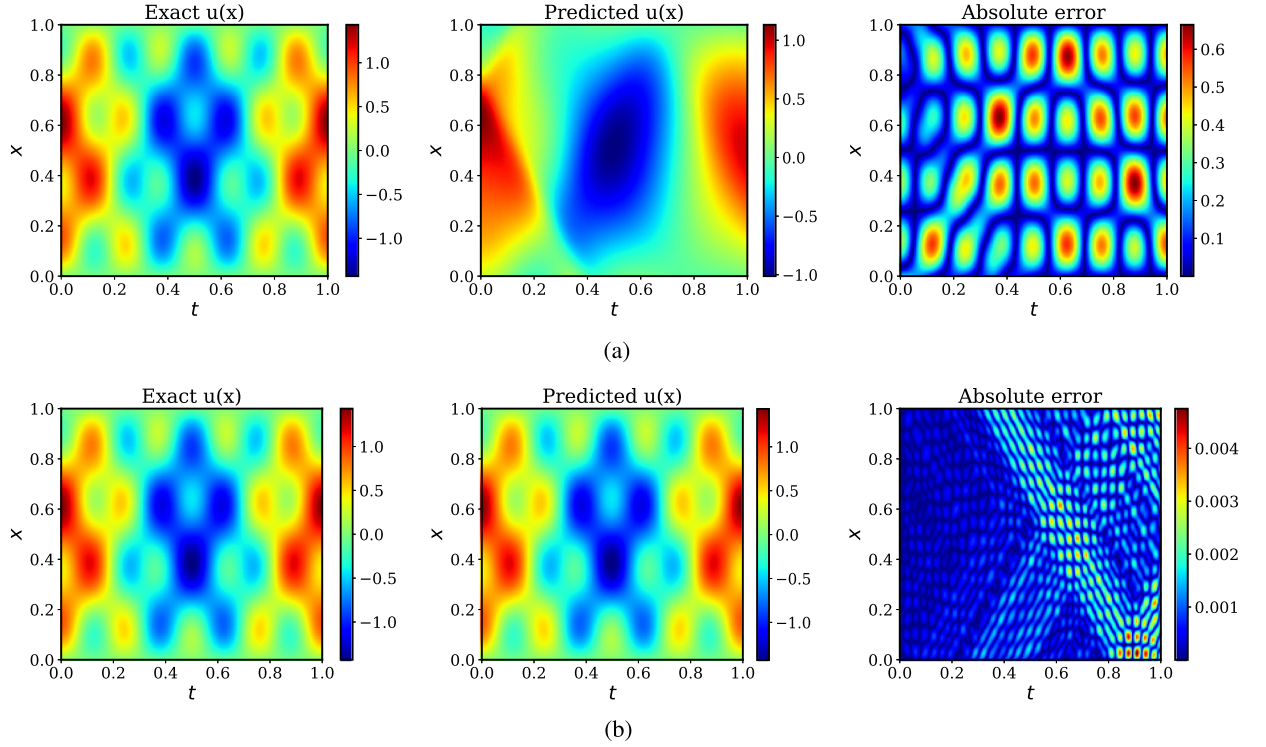


Fig. 7. One-dimensional wave equation: Eigenvalues of K_u , K_{u_t} and K_r at different snapshots during training, sorted in descending order.

of $u_t(\mathbf{x}_{u_t}; \theta)$ and $\mathcal{N}[u](\mathbf{x}_r; \theta)$ converge much faster than the loss of boundary conditions, and, therefore, introduce a severe discrepancy in the convergence rate of each different term in the loss, causing this standard PINNs model to collapse. To verify our hypothesis, we also train the same network using Algorithm 1 with the following generalized updating rule for hyper-parameters λ_u , λ_{u_t} and λ_r

$$\lambda_u = \frac{\text{Tr}(\mathbf{K}_u) + \text{Tr}(\mathbf{K}_{u_t}) + \text{Tr}(\mathbf{K}_r)}{\text{Tr}(\mathbf{K}_u)} \quad (7.13)$$

$$\lambda_{u_t} = \frac{\text{Tr}(\mathbf{K}_u) + \text{Tr}(\mathbf{K}_{u_t}) + \text{Tr}(\mathbf{K}_r)}{\text{Tr}(\mathbf{K}_{u_t})} \quad (7.14)$$

$$\lambda_r = \frac{\text{Tr}(\mathbf{K}_u) + \text{Tr}(\mathbf{K}_{u_t}) + \text{Tr}(\mathbf{K}_r)}{\text{Tr}(\mathbf{K}_r)}. \quad (7.15)$$

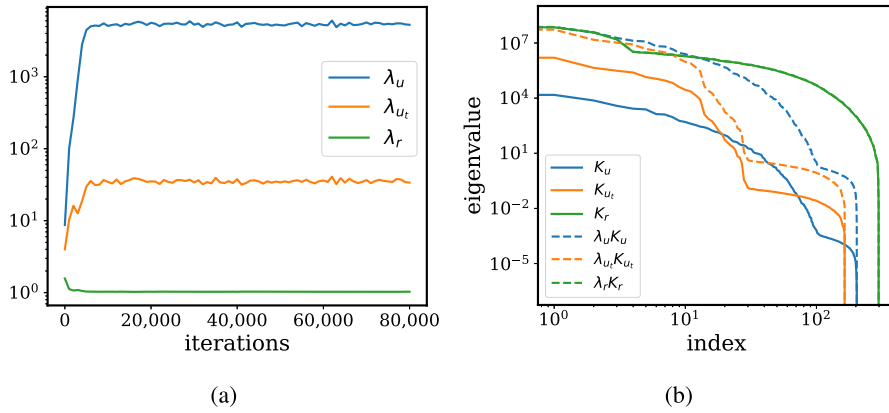


Fig. 8. One-dimensional wave equation: (a) The evolution of hyper-parameters λ_u, λ_{u_t} and λ_r during training of a five-layer deep fully-connected neural network with 500 neurons per layer using Algorithm 1. (b) The eigenvalues of K_u, K_{u_t}, K_r and $\lambda_u K_u, \lambda_{u_t} K_{u_t}, \lambda_r K_r$ at last step of training.

In particular, we update these weights every 1,000 training iterations, hence the extra computational costs compared to a standard PINNs approach is negligible. The results of this experiment are shown in Fig. 6b, from which one can easily see that the predicted solution obtained using the proposed adaptive training scheme achieves excellent agreement with the ground truth and the relative L^2 error is $1.73e-3$. To quantify the effect of the hyper-parameters λ_u, λ_{u_t} and λ_r on the NTK, we also compare the eigenvalues of K_u, K_{u_t} and K_r multiplied with or without the hyper-parameters at last step of gradient descent. As it can be seen in Fig. 8b, the discrepancy of the convergence rate of different components in total training errors is considerably resolved. Furthermore, Fig. 8a presents the change of weights during training and we can see that λ_u, λ_{u_t} increase rapidly and then remain almost fixed while λ_r is near 1 for all time. So we may conclude that the overall training process using Algorithm 1 is stable.

8. Discussion

This work has produced a novel theoretical understanding of physics-informed neural networks by deriving and analyzing their limiting neural tangent kernel. Specifically, we first show that infinitely wide physics-informed neural networks under the NTK parameterization converge to Gaussian processes. Furthermore, we derive NTK of PINNs and show that, under suitable assumptions, it converges to a deterministic kernel and barely changes during training as the width of the network grows to infinity. To provide further insight, we analyze the training dynamics of fully-connected PINNs through the lens of their NTK and show that not only they suffer from spectral bias, but they also exhibit a discrepancy in the convergence rate among the different loss components contributing to the total training error. To resolve this discrepancy, we propose a novel algorithm such that the coefficients of different terms in a PINNs' loss function can be dynamically updated according to balance the average convergence rate of different components in the total training error. Finally, we carry out a series of numerical experiments to verify our theory and validate the effectiveness of the proposed algorithms.

Although this work takes an essential step towards understanding PINNs and their training dynamics, there are many open questions worth exploring. Can the proposed NTK theory for PINNs be extended fully-connected networks with multiple hidden layers, nonlinear equations, as well as the neural network architectures such as convolutional neural networks, residual networks, etc.? To which extend these architecture suffer from spectral bias or exhibit similar discrepancies in their convergence rate? In a parallel thrust, it is well-known that PINNs perform much better for inverse problems than for forward problems, such as the ones considered in this work. Can we incorporate the current theory to analyze inverse problems and explain they are better suited to PINNs? Moreover, going beyond vanilla gradient descent dynamics, how do the training dynamics of PINNs and their corresponding NTK evolve via gradient descent with momentum (e.g. Adam [47])? In practice, despite some improvements in the performance of PINNs brought by assigning appropriate weights to the loss function, we emphasize that such methods cannot change the distribution of eigenvalues of the NTK and, thus, cannot directly resolve spectral bias. Apart from this, assigning weights may result in indefinite kernels which can have imaginary eigenvalues and thus yield unstable training processes. Therefore, can we come up with better methodologies to resolve spectral bias using specialized network architectures, loss functions, etc.? In a broad sense, PINNs can be regarded as a special multi-task learning problem in which a neural network is asked to simultaneously fit the observed data and minimize a PDE residual. It is then natural to ask: does the proposed NTK theory hold for general multi-task learning problems? Can other heuristic methods for multi-task learning [48–50] be analyzed and improved under this setting? We believe that answering these questions not only paves a new way to better understand PINNs and its training dynamics, but also opens a new door for developing scientific machine learning algorithms with provable convergence guarantees, as needed for many critical applications in computational science and engineering.

CRedit authorship contribution statement

Sifan Wang: Conceptualization, Methodology, Implementation, Numerical experiments, Visualization, Draft Preparation.

Xinling Yu: Numerical experiments.

Paris Perdikaris: Conceptualization, Methodology, Supervision, Funding, Writing-Reviewing and Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work received support from DOE grant DE-SC0019116, AFOSR grant FA9550-20-1-0060, and DOE-ARPA grant DE-AR0001201.

Appendix A. Proof of Lemma 3.1

Proof. Recall that for given training data $\{\mathbf{x}_b^i, g(\mathbf{x}_b^i)\}_{i=1}^{N_b}, \{\mathbf{x}_r^i, f(\mathbf{x}_r^i)\}_{i=1}^{N_r}$, the loss function is given by

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}) &= \mathcal{L}_b(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) \\ &= \frac{1}{2} \sum_{i=1}^{N_b} |u(\mathbf{x}_b^i; \boldsymbol{\theta}) - g(\mathbf{x}_b^i)|^2 + \frac{1}{2} \sum_{i=1}^{N_r} |r(\mathbf{x}_r^i; \boldsymbol{\theta})|^2.\end{aligned}$$

Now let us consider the corresponding gradient flow

$$\frac{d\boldsymbol{\theta}}{dt} = -\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = -\left[\sum_{i=1}^{N_b} (u(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b^i)) \frac{\partial u}{\partial \boldsymbol{\theta}}(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) + \sum_{i=1}^{N_r} (\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r^i)) \frac{\partial \mathcal{N}[u]}{\partial \boldsymbol{\theta}}(\mathbf{x}_r^i; \boldsymbol{\theta}(t)) \right].$$

It follows that for $0 \leq j \leq N_b$,

$$\begin{aligned}\frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{dt} &= \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \cdot \frac{d\boldsymbol{\theta}}{dt} \\ &= -\frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \cdot \left[\sum_{i=1}^{N_b} (u(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b^i)) \frac{\partial u}{\partial \boldsymbol{\theta}}(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) + \sum_{i=1}^{N_r} (\mathcal{L}(\mathbf{x}_r^i; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r^i)) \frac{\partial \mathcal{N}[u]}{\partial \boldsymbol{\theta}}(\mathbf{x}_r^i; \boldsymbol{\theta}(t)) \right] \\ &= -\sum_{i=1}^{N_b} (u(\mathbf{x}_b^i; \boldsymbol{\theta}) - g(\mathbf{x}_b^i)) \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ &\quad - \sum_{i=1}^{N_r} (\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}) - f(\mathbf{x}_r^i)) \left\langle \frac{\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle.\end{aligned}$$

Similarly,

$$\begin{aligned}\frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{dt} &= \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \cdot \frac{d\boldsymbol{\theta}}{dt} \\ &= \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \cdot \left[\sum_{i=1}^{N_b} (u(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b^i)) \frac{\partial u}{\partial \boldsymbol{\theta}}(\mathbf{x}_b^i; \boldsymbol{\theta}(t)) + \sum_{i=1}^{N_r} (\mathcal{L}(\mathbf{x}_r^i; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r^i)) \frac{\partial \mathcal{N}[u]}{\partial \boldsymbol{\theta}}(\mathbf{x}_r^i; \boldsymbol{\theta}(t)) \right] \\ &= -\sum_{i=1}^{N_b} (u(\mathbf{x}_b^i; \boldsymbol{\theta}) - g(\mathbf{x}_b^i)) \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ &\quad - \sum_{i=1}^{N_r} (\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}) - f(\mathbf{x}_r^i)) \left\langle \frac{\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle.\end{aligned}$$

Then we can rewrite the above equations as

$$\frac{du(\mathbf{x}_b; \boldsymbol{\theta}(t))}{dt} = -\mathbf{K}_{uu}(t) \cdot (u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b)) - \mathbf{K}_{ur}(t) \cdot (\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r)) \quad (\text{A.1})$$

$$\frac{d\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t))}{dt} = -\mathbf{K}_{ru}(t) \cdot (u(\mathbf{x}_b; \boldsymbol{\theta}(t)) - g(\mathbf{x}_b)) - \mathbf{K}_{rr}(t) \cdot (\mathcal{N}[u](\mathbf{x}_r; \boldsymbol{\theta}(t)) - f(\mathbf{x}_r)), \quad (\text{A.2})$$

where $\mathbf{K}_{ru}(t) = \mathbf{K}_{ur}^T(t)$ and the (i, j) -th entries are given by

$$\begin{aligned} (\mathbf{K}_{uu})_{ij}(t) &= \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du(\mathbf{x}_b^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ (\mathbf{K}_{ur})_{ij}(t) &= \left\langle \frac{du(\mathbf{x}_b^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \\ (\mathbf{K}_{rr})_{ij}(t) &= \left\langle \frac{d\mathcal{N}[u](\mathbf{x}_r^i; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{d\mathcal{N}[u](\mathbf{x}_r^j; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle. \quad \square \end{aligned}$$

Appendix B. Proof of Theorem 4.1

Proof. Recall equation (4.3) and that all weights and biases are initialized by independent standard Gaussian distributions. Then by the central limit theorem, we have

$$\begin{aligned} u_{xx}(x; \boldsymbol{\theta}) &= \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \left[\ddot{\sigma}(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right] \\ &= \frac{1}{\sqrt{N}} \sum_{k=1}^N \mathbf{W}_k^{(1)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) (\mathbf{W}_k^{(0)})^2 \\ &\xrightarrow{\mathcal{D}} \mathcal{N}(0, \Sigma(x)) \triangleq Y(x), \end{aligned}$$

as $N \rightarrow \infty$, where $\mathcal{D}$ denotes convergence in distribution and $Y(x)$ is a centered Gaussian random variable with covariance

$$\Sigma(x) = \text{Var}[\mathbf{W}_k^{(1)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) (\mathbf{W}_k^{(0)})^2].$$

Since $\ddot{\sigma}$ is bounded, we may assume that $|\ddot{\sigma}| \leq C$. Then we have

$$\begin{aligned} \sup_N \mathbb{E} \left[|u_{xx}(x; \boldsymbol{\theta})|^2 \right] &= \sup_N \mathbb{E} \left[\mathbb{E} \left[|u_{xx}(x; \boldsymbol{\theta})|^2 \mid \mathbf{W}^{(0)}, \mathbf{b}^{(0)} \right] \right] \\ &= \sup_N \mathbb{E} \left[\frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(0)})^4 (\ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}))^2 \right] \\ &\leq C \mathbb{E}[(\mathbf{W}_k^{(0)})^4] < \infty. \end{aligned}$$

This implies that $u_{xx}(x; \boldsymbol{\theta})$ is uniformly integrable with respect to N . Now for any given point x, x' , we have

$$\begin{aligned} \Sigma_{xx}^{(1)}(x, x') &\triangleq \mathbb{E}[Y(x)Y(x')] = \lim_{N \rightarrow \infty} \mathbb{E}[u_{xx}(x, \boldsymbol{\theta})u_{xx}(x', \boldsymbol{\theta})] \\ &= \lim_{N \rightarrow \infty} \mathbb{E} \left[\mathbb{E}[u_{xx}(x, \boldsymbol{\theta})u_{xx}(x', \boldsymbol{\theta}) \mid \mathbf{W}^{(0)}, \mathbf{b}^{(0)}] \right] \\ &= \lim_{N \rightarrow \infty} \mathbb{E} \left[\frac{1}{N} \left(\ddot{\sigma}(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right)^T \left(\ddot{\sigma}(\mathbf{W}^{(0)}x' + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right) \right] \\ &= \lim_{N \rightarrow \infty} \mathbb{E} \left[\frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(0)})^4 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] \\ &= \mathbb{E}_{u, v \sim \mathcal{N}(0, 1)} \left[u^4 \ddot{\sigma}(ux + v) \ddot{\sigma}(ux' + v) \right]. \end{aligned}$$

This concludes the proof. $\square$

Appendix C. Proof of Theorem 4.3

Proof. To warm up, we first compute $\mathbf{K}_{uu}(0)$ and its infinite width limit, which is already covered in [33]. By the definition of $\mathbf{K}_{uu}(0)$, for any two given input x, x' we have

$$\mathbf{K}_{uu}(0) = \left\langle \frac{du(x; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}}, \frac{du(x'; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}} \right\rangle.$$

Recall that

$$u(x; \boldsymbol{\theta}) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \sigma(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)} = \frac{1}{\sqrt{N}} \sum_{k=1}^N \mathbf{W}_k^{(1)} \sigma(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) + \mathbf{b}^{(1)},$$

and $\boldsymbol{\theta} = (\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \mathbf{b}^{(0)}, \mathbf{b}^{(1)})$. Then we have

$$\begin{aligned} \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x \\ \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{\sqrt{N}} \sigma(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \\ \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \\ \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{b}^{(1)}} &= 1. \end{aligned}$$

Then by the law of large numbers we have

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(0)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{N} \sum_{k=1}^N \left[\mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x \right] \cdot \left[\mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)})x' \right] \\ &= \frac{1}{N} \left(\sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right) (xx') \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(1)})^2 \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] (xx') \\ &= \mathbb{E} \left[(\mathbf{W}_k^{(1)})^2 \right] \mathbb{E} \left[\dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] (xx') = \dot{\Sigma}^{(1)}(x, x') (xx'), \end{aligned}$$

as $N \rightarrow \infty$, where $\dot{\Sigma}^{(1)}(x, x')$ is defined in equation (2.5).

Moreover,

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{N} \sum_{k=1}^N \sigma(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \sigma(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[\sigma(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \sigma(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] = \Sigma^{(1)}(x, x'), \end{aligned}$$

as $N \rightarrow \infty$, where $\Sigma^{(1)}(x, x')$ is defined in equation (2.4).

Also,

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{N} \sum_{k=1}^N \left[(\mathbf{W}_k^{(1)})^2 \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[\dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \dot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] = \dot{\Sigma}^{(1)}(x, x'). \end{aligned}$$

Then plugging all these together we obtain

$$\begin{aligned} \mathbf{K}_{uu}(0) &= \left\langle \frac{du(x; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}}, \frac{du(x'; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}} \right\rangle \\ &= \sum_{l=0}^1 \sum_{k=1}^N \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(l)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(l)}} + \sum_{k=1}^N \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} + \frac{\partial u(x; \boldsymbol{\theta})}{\partial \mathbf{b}^{(1)}} \frac{\partial u(x'; \boldsymbol{\theta})}{\partial \mathbf{b}^{(1)}} \end{aligned}$$

$$\xrightarrow{\mathcal{P}} \dot{\Sigma}^{(1)}(x, x')(xx') + \Sigma^{(1)}(x, x') + \dot{\Sigma}^{(1)}(x, x') + 1 \triangleq \Theta_{uu}^{(1)},$$

as $N \rightarrow \infty$. This formula is also consistent with equation (2.8).

Next, we compute $\mathbf{K}_{rr}(0)$. To this end, recall that

$$u_{xx}(x; \theta) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \left[\ddot{\sigma}(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right] = \frac{1}{\sqrt{N}} \sum_{k=1}^N \mathbf{W}_k^{(1)} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}).$$

It is then easy to compute that

$$\begin{aligned} \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)} \left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x + 2\ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \right] \\ \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{\sqrt{N}} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \\ \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}), \end{aligned}$$

where $\ddot{\sigma}$ denotes third order derivative of σ . Then we have

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(0)}} \frac{\partial u_{xx}(x'; \theta)}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{N} \sum_{k=1}^N \left(\mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)} \right)^2 \left(\left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x + 2\ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \right] \right. \\ &\quad \cdot \left. \left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)})x' + 2\ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] \right) \\ &= I_1 + I_2 + I_3 + I_4, \end{aligned}$$

where

$$\begin{aligned} I_1 &= \frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 (\mathbf{W}_k^{(0)})^4 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)})x' \\ I_2 &= \frac{2}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 (\mathbf{W}_k^{(0)})^3 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)})x \\ I_3 &= \frac{2}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 (\mathbf{W}_k^{(0)})^3 \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x' \\ I_4 &= \frac{4}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}). \end{aligned}$$

By the law of large numbers, letting $N \rightarrow \infty$ gives

$$\begin{aligned} I_1 &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^4 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] xx' := J_1 \\ I_2 &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^3 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] x := J_2 \\ I_3 &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^3 \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \right] x' := J_3 \\ I_4 &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}x' + \mathbf{b}_k^{(0)}) \right] := J_4. \end{aligned}$$

In conclusion we have

$$\sum_{k=1}^N \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(0)}} \frac{\partial u_{xx}(x'; \theta)}{\partial \mathbf{W}_k^{(0)}} \xrightarrow{\mathcal{P}} J_1 + J_2 + J_3 + J_4 := A_{rr}.$$

Moreover,

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} \frac{\partial u_{xx}(\mathbf{x}'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(0)})^4 \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^4 \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \right] := B_{rr}, \end{aligned}$$

and

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} \frac{\partial u_{xx}(\mathbf{x}'; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 (\mathbf{W}_k^{(0)})^4 \left[\ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \right] \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^4 (\ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)})) \right] := C_{rr}. \end{aligned}$$

Now, recall that

$$\mathbf{K}_{rr}(0) = \left\langle \frac{du_{xx}(\mathbf{x}; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}}, \frac{du_{xx}(\mathbf{x}'; \boldsymbol{\theta}(0))}{d\boldsymbol{\theta}} \right\rangle.$$

Thus we can conclude that as $N \rightarrow \infty$,

$$\mathbf{K}_{rr}(0) \xrightarrow{\mathcal{P}} A_{rr} + B_{rr} + C_{rr} := \Theta_{rr}(\mathbf{x}, \mathbf{x}').$$

Finally, recall that $\mathbf{K}_{ur}(\mathbf{x}, \mathbf{x}') = \mathbf{K}_{ru}(\mathbf{x}', \mathbf{x})$. So it suffices to compute $\mathbf{K}_{ur}(\mathbf{x}, \mathbf{x}')$ and its limit. To this end, recall that

$$\mathbf{K}_{ur}(\mathbf{x}, \mathbf{x}') = \left\langle \frac{du(\mathbf{x}; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du_{xx}(\mathbf{x}'; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle.$$

Then letting $N \rightarrow \infty$ gives

$$\begin{aligned} &\sum_{k=1}^N \frac{\partial u(\mathbf{x}; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(0)}} \frac{\partial u_{xx}(\mathbf{x}'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(0)}} \\ &= \frac{1}{N} \sum_{k=1}^N (\mathbf{W}_k^{(1)})^2 \mathbf{W}_k^{(0)} \left[\dot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \mathbf{x} \right] \cdot \left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \mathbf{x}' + 2\ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \right] \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^2 \dot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \right] (\mathbf{x}\mathbf{x}') + 2\mathbb{E} \left[\mathbf{W}_k^{(0)} \dot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \right] \mathbf{x} \\ &:= A_{ur}, \end{aligned}$$

and

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u(\mathbf{x}; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} \frac{\partial u_{xx}(\mathbf{x}'; \boldsymbol{\theta})}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{N} \sum_{k=1}^N \left[(\mathbf{W}_k^{(0)})^2 \sigma(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \right] \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^2 \sigma(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \right] := B_{ur}, \end{aligned}$$

and

$$\begin{aligned} \sum_{k=1}^N \frac{\partial u(\mathbf{x}; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} \frac{\partial u_{xx}(\mathbf{x}'; \boldsymbol{\theta})}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{N} \sum_{k=1}^N \left[\mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)} \right]^2 \cdot \left[\dot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}) \right] \\ &\xrightarrow{\mathcal{P}} \mathbb{E} \left[(\mathbf{W}_k^{(0)})^2 \cdot (\dot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x} + \mathbf{b}_k^{(0)}) \cdot \ddot{\sigma}(\mathbf{W}_k^{(0)}\mathbf{x}' + \mathbf{b}_k^{(0)}))^2 \right] := C_{ur}. \end{aligned}$$

As a result, we obtain

$$\mathbf{K}_{ur}(\mathbf{x}, \mathbf{x}') = \left\langle \frac{du(\mathbf{x}; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}}, \frac{du_{xx}(\mathbf{x}'; \boldsymbol{\theta}(t))}{d\boldsymbol{\theta}} \right\rangle \xrightarrow{\mathcal{P}} A_{ur} + B_{ur} + C_{ur} : \Theta_{ur}^{(1)},$$

as $N \rightarrow \infty$. This concludes the proof. $\square$

Appendix D. Proof of Theorem 4.4

Before we prove the main theorem, we need to prove a series of lemmas.

Lemma D.1. *Under the setting of Theorem 4.4, for $l = 0, 1$, we have*

$$\begin{aligned} \sup_{t \in [0, T]} \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}} \right\|_{\infty} &= \mathcal{O}\left(\frac{1}{\sqrt{N}}\right), \\ \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}}{\partial \mathbf{W}^{(l)}} \right\|_{\infty} &= \mathcal{O}\left(\frac{1}{\sqrt{N}}\right), \\ \sup_{t \in [0, T]} \left\| \frac{\partial u}{\partial \mathbf{b}^{(0)}} \right\|_{\infty} &= \mathcal{O}\left(\frac{1}{\sqrt{N}}\right), \\ \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}}{\partial \mathbf{b}^{(0)}} \right\|_{\infty} &= \mathcal{O}\left(\frac{1}{\sqrt{N}}\right). \end{aligned}$$

Proof. For the given model problem, recall that

$$u(x; \theta) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \sigma(\mathbf{W}^{(0)}(t)x + \mathbf{b}^{(0)}) + \mathbf{b}^{(1)},$$

and

$$\begin{aligned} \frac{\partial u(x; \theta)}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x \\ \frac{\partial u(x; \theta)}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{\sqrt{N}} \sigma(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \\ \frac{\partial u(x; \theta)}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}). \end{aligned}$$

Then by assumptions (i), (ii), and given that Ω is bounded, we have

$$\begin{aligned} \sup_{t \in [0, T]} \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}} \right\|_{\infty} &\leq \frac{C}{\sqrt{N}}, \quad l = 0, 1. \\ \sup_{t \in [0, T]} \left\| \frac{\partial u}{\partial \mathbf{b}^{(0)}} \right\|_{\infty} &\leq \frac{C}{\sqrt{N}}. \end{aligned}$$

Also,

$$u_{xx}(x; \theta) = \frac{1}{\sqrt{N}} \mathbf{W}^{(1)} \cdot \left[\ddot{\sigma}(\mathbf{W}^{(0)}x + \mathbf{b}^{(0)}) \odot \mathbf{W}^{(0)} \odot \mathbf{W}^{(0)} \right] = \frac{1}{\sqrt{N}} \sum_{k=1}^N \mathbf{W}_k^{(1)} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}),$$

and

$$\begin{aligned} \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)} \left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)})x + 2\dot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \right] \\ \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(1)}} &= \frac{1}{\sqrt{N}} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}) \\ \frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{b}_k^{(0)}} &= \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)}x + \mathbf{b}_k^{(0)}). \end{aligned}$$

Again, using assumptions (i), (ii) gives

$$\begin{aligned} \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}}{\partial \mathbf{W}^{(l)}} \right\|_{\infty} &\leq \frac{C^4}{\sqrt{N}}, \quad l = 0, 1. \\ \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}}{\partial \mathbf{b}^{(0)}} \right\|_{\infty} &\leq \frac{C^4}{\sqrt{N}}. \end{aligned}$$

This completes the proof. $\square$

Lemma D.2. Under the setting of Theorem 4.4, we have

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{W}^{(l)}(t) - \mathbf{W}^{(l)}(0) \right) \right\|_2 = 0, \quad l = 0, 1. \quad (\text{D.1})$$

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{b}^{(0)}(t) - \mathbf{b}^{(0)}(0) \right) \right\|_2 = 0. \quad (\text{D.2})$$

Proof. Recall that the loss function for the model problem (4.1) is given by

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_b(\boldsymbol{\theta}) + \mathcal{L}_r(\boldsymbol{\theta}) = \frac{1}{2} \sum_{i=1}^{N_b} |u(x_b^i; \boldsymbol{\theta}) - g(x_b^i)|^2 + \frac{1}{2} \sum_{i=1}^{N_r} |u_{xx}(x_r^i; \boldsymbol{\theta}) - f(x_r^i)|^2.$$

Consider minimizing the loss function $\mathcal{L}(\boldsymbol{\theta})$ by gradient descent with an infinitesimally small learning rate:

$$\frac{d\boldsymbol{\theta}}{dt} = -\nabla \mathcal{L}(\boldsymbol{\theta}).$$

This implies that

$$\begin{aligned} \frac{d\mathbf{W}^{(l)}}{dt} &= -\frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{W}^{(l)}}, \quad l = 0, 1, \\ \frac{d\mathbf{b}^{(0)}}{dt} &= -\frac{\partial \mathcal{L}(\boldsymbol{\theta})}{\partial \mathbf{b}^{(0)}}. \end{aligned}$$

Then we have

$$\begin{aligned} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{W}^{(l)}(t) - \mathbf{W}^{(l)}(0) \right) \right\|_2 &= \left\| \frac{1}{\sqrt{N}} \int_0^t \frac{d\mathbf{W}^{(l)}(\tau)}{d\tau} d\tau \right\|_2 = \left\| \frac{1}{\sqrt{N}} \int_0^t \frac{\partial \mathcal{L}(\boldsymbol{\theta}(\tau))}{\partial \mathbf{W}^{(l)}} d\tau \right\|_2 \\ &= \left\| \frac{1}{\sqrt{N}} \int_0^t \left[\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) + \sum_{i=1}^{N_r} (u_{xx}(x_r^i; \boldsymbol{\theta}(\tau)) - f(x_r^i)) \frac{\partial u_{xx}}{\partial \mathbf{W}^{(l)}}(x_r^i; \boldsymbol{\theta}(\tau)) \right] d\tau \right\|_2 \\ &\leq I_1^{(l)} + I_2^{(l)}, \end{aligned}$$

where

$$\begin{aligned} I_1^{(l)} &= \left\| \frac{1}{\sqrt{N}} \int_0^t \left[\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right] d\tau \right\|_2 \\ I_2^{(l)} &= \left\| \frac{1}{\sqrt{N}} \int_0^t \left[\sum_{i=1}^{N_r} (u_{xx}(x_r^i; \boldsymbol{\theta}(\tau)) - f(x_r^i)) \frac{\partial u_{xx}}{\partial \mathbf{W}^{(l)}}(x_r^i; \boldsymbol{\theta}(\tau)) \right] d\tau \right\|_2. \end{aligned}$$

We first process to estimate $I_1^{(l)}$ as

$$\begin{aligned} I_1^{(l)} &\leq \frac{1}{\sqrt{N}} \int_0^t \left\| \left[\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right] \right\|_2 d\tau \\ &= \frac{1}{\sqrt{N}} \int_0^t \sqrt{\sum_{k=1}^N \left(\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \frac{\partial u}{\partial \mathbf{W}_k^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right)^2} d\tau \\ &\leq \frac{1}{\sqrt{N}} \int_0^t \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right\|_\infty \sqrt{\sum_{k=1}^N \left(\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \right)^2} d\tau \end{aligned}$$

$$= \frac{1}{\sqrt{N}} \int_0^T \sqrt{N} \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right\|_{\infty} \cdot \left| \sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \right| d\tau.$$

Thus, by assumptions and Lemma D.1, for $l = 0, 1$ we have

$$\begin{aligned} \sup_{t \in [0, T]} I_1^{(l)} &= \sup_{t \in [0, T]} \int_0^T \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right\|_{\infty} \cdot \left| \sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \right| d\tau \\ &\leq \frac{C}{\sqrt{N}} \rightarrow 0, \quad \text{as } N \rightarrow \infty. \end{aligned}$$

Similarly,

$$\begin{aligned} \sup_{t \in [0, T]} I_2^{(l)} &\leq \sup_{t \in [0, T]} \leq \frac{1}{\sqrt{N}} \int_0^T \left\| \frac{\partial u}{\partial \mathbf{W}^{(l)}}(x_b^i; \boldsymbol{\theta}(\tau)) \right\|_{\infty} \sqrt{\sum_{k=1}^N \left(\sum_{i=1}^{N_b} (u(x_b^i; \boldsymbol{\theta}(\tau)) - g(x_b^i)) \right)^2} d\tau \\ &= \frac{1}{\sqrt{N}} \int_0^T \sqrt{N} \left\| \frac{\partial u_{xx}}{\partial \mathbf{W}^{(l)}}(x_r^i; \boldsymbol{\theta}(\tau)) \right\|_{\infty} \cdot \left| \sum_{i=1}^{N_r} (u_{xx}(x_r^i; \boldsymbol{\theta}(\tau)) - f(x_r^i)) \right| d\tau \\ &\leq \frac{C^4}{\sqrt{N}} \rightarrow 0, \quad \text{as } N \rightarrow \infty. \end{aligned}$$

Plugging these together, we obtain

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{W}^{(l)}(t) - \mathbf{W}^{(l)}(0) \right) \right\|_2 \leq \lim_{N \rightarrow \infty} \sup_{t \in [0, T]} I_1^{(l)} + I_2^{(l)} = 0,$$

for $l = 1, 2$. Similarly, applying the same strategy to $\mathbf{b}^{(0)}$ we can show

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{b}^{(0)}(t) - \mathbf{b}^{(0)}(0) \right) \right\|_2 = 0.$$

This concludes the proof. $\square$

Lemma D.3. Under the setting of Theorem 4.4, we have

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\sigma^{(k)}(\mathbf{W}^{(0)}(t)x + \mathbf{b}^{(0)}(t)) - \sigma^{(k)}(\mathbf{W}^{(0)}(0)x + \mathbf{b}^{(0)}(0)) \right) \right\|_2 = 0, \quad (\text{D.3})$$

for $k = 0, 1, 2, 3$, where $\sigma^{(k)}$ denotes the k -th order derivative of σ .

Proof. By the mean-value theorem for vector-valued function and Lemma D.2, there exists ξ

$$\begin{aligned} &\left\| \frac{1}{\sqrt{N}} \left(\sigma^{(k)}(\mathbf{W}^{(0)}(t)x + \mathbf{b}^{(0)}(t)) - \sigma^{(k)}(\mathbf{W}^{(0)}(0)x + \mathbf{b}^{(0)}(0)) \right) \right\|_2 \\ &\leq \left\| \sigma^{(k+1)}(\xi) \right\| \left\| \frac{1}{\sqrt{N}} \left(\mathbf{W}^{(0)}(t)x + \mathbf{b}^{(0)}(t) - \mathbf{W}^{(0)}(0)x + \mathbf{b}^{(0)}(0) \right) \right\|_2 \\ &\leq C \left\| \frac{1}{\sqrt{N}} \left(\mathbf{W}^{(0)}(t) - \mathbf{W}^{(0)}(0) \right) \right\|_2 + C \left\| \frac{1}{\sqrt{N}} \left(\mathbf{b}^{(0)}(t) - \mathbf{b}^{(0)}(0) \right) \right\|_2 \\ &\rightarrow 0, \end{aligned}$$

as $N \rightarrow \infty$. Here we use the assumption that $\sigma^{(k)}$ is bounded for $k = 0, 1, 2, 3, 4$. This concludes the proof. $\square$

Lemma D.4. Under the setting of Theorem 4.4, we have

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \theta(t))}{\partial \theta} - \frac{\partial u(x; \theta(0))}{\partial \theta} \right\|_2 \quad (\text{D.4})$$

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(x; \theta(t))}{\partial \theta} - \frac{\partial u_{xx}(x; \theta(0))}{\partial \theta} \right\|_2. \quad (\text{D.5})$$

Proof. Recall that

$$\frac{\partial u(x; \theta)}{\partial \mathbf{W}_k^{(0)}} = \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)}) x$$

$$\frac{\partial u(x; \theta)}{\partial \mathbf{W}_k^{(1)}} = \frac{1}{\sqrt{N}} \sigma(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)})$$

$$\frac{\partial u(x; \theta)}{\partial \mathbf{b}_k^{(0)}} = \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \dot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)})$$

$$\frac{\partial u(x; \theta)}{\partial \mathbf{b}^{(1)}} = 1.$$

To simplify notation, let us define

$$\mathbf{A}(t) = [\mathbf{W}^{(1)}(t)]^T$$

$$\mathbf{B}(t) = \dot{\sigma}(\mathbf{W}^{(0)}(t)x + \mathbf{b}^{(0)}(t))x.$$

Then by assumption (i) Lemma D.2 D.3, we have

$$\begin{aligned} & \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \theta(t))}{\partial \mathbf{W}^{(0)}} - \frac{\partial u(x; \theta(0))}{\partial \mathbf{W}^{(0)}} \right\|_2 \\ &= \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) \odot \mathbf{B}(t) - \mathbf{A}(0) \odot \mathbf{B}(0)) \right\|_2 \\ &\leq \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) - \mathbf{A}(0)) \odot \mathbf{B}(t) \right\|_2 + \left\| \frac{1}{\sqrt{N}} \mathbf{A}(0) \odot (\mathbf{B}(t) - \mathbf{B}(0)) \right\|_2 \\ &\leq \sup_{t \in [0, T]} \|\mathbf{B}(t)\|_\infty \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) - \mathbf{A}(0)) \right\|_2 + \sup_{t \in [0, T]} \|\mathbf{A}(0)\|_\infty \left\| \frac{1}{\sqrt{N}} (\mathbf{B}(t) - \mathbf{B}(0)) \right\|_2 \\ &\rightarrow 0, \end{aligned}$$

as $N \rightarrow \infty$. Here $\odot$ denotes point-wise multiplication.

Similarly, we can show that

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \theta(t))}{\partial \mathbf{W}^{(1)}} - \frac{\partial u(x; \theta(0))}{\partial \mathbf{W}^{(1)}} \right\|_2 = 0,$$

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \theta(t))}{\partial \mathbf{b}^{(0)}} - \frac{\partial u(x; \theta(0))}{\partial \mathbf{b}^{(0)}} \right\|_2 = 0.$$

Thus, we conclude that

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \theta(t))}{\partial \theta} - \frac{\partial u(x; \theta(0))}{\partial \theta} \right\|_2 = 0.$$

Now for u_{xx} , we know that

$$\frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(0)}} = \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} \mathbf{W}_k^{(0)} \left[\mathbf{W}_k^{(0)} \ddot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)}) x + 2 \dot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)}) \right]$$

$$\frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{W}_k^{(1)}} = \frac{1}{\sqrt{N}} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)})$$

$$\frac{\partial u_{xx}(x; \theta)}{\partial \mathbf{b}_k^{(0)}} = \frac{1}{\sqrt{N}} \mathbf{W}_k^{(1)} (\mathbf{W}_k^{(0)})^2 \ddot{\sigma}(\mathbf{W}_k^{(0)} x + \mathbf{b}_k^{(0)}).$$

Then for $\mathbf{W}^{(0)}$, again we define

$$\begin{aligned}\mathbf{A}(t) &= [\mathbf{W}^{(1)}]^T \\ \mathbf{B}(t) &= \mathbf{W}^{(0)} \\ \mathbf{C}(t) &= \ddot{\sigma}(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)})\mathbf{x} \\ \mathbf{D}(t) &= 2\ddot{\sigma}(\mathbf{W}^{(0)}\mathbf{x} + \mathbf{b}^{(0)}).\end{aligned}$$

Then,

$$\begin{aligned}& \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta}(t))}{\partial \mathbf{W}^{(0)}} - \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta}(0))}{\partial \mathbf{W}^{(0)}} \right\|_2 \\&= \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{A}(t) \odot \mathbf{B}(t) \odot [\mathbf{B}(t) \odot \mathbf{C}(t) + \mathbf{D}(t)] - \mathbf{A}(0) \odot \mathbf{B}(0) \odot [\mathbf{B}(0) \odot \mathbf{C}(0) + \mathbf{D}(0)] \right) \right\|_2 \\&\leq \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{A}(t) \odot \mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) - \mathbf{A}(0) \odot \mathbf{B}(0) \odot \mathbf{B}(0) \odot \mathbf{C}(0) \right) \right\|_2 \\&+ \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{A}(t) \odot \mathbf{B}(t) \odot \mathbf{D}(t) - \mathbf{A}(0) \odot \mathbf{B}(0) \odot \mathbf{D}(0) \right) \right\|_2 \\&:= I_1 + I_2.\end{aligned}$$

For I_1 , we have

$$\begin{aligned}& \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{A}(t) \odot \mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) - \mathbf{A}(0) \odot \mathbf{B}(0) \odot \mathbf{B}(0) \odot \mathbf{C}(0) \right) \right\|_2 \\&\leq \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left([\mathbf{A}(t) - \mathbf{A}(0)] \odot \mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) \right) \right\|_2 \\&+ \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} \left(\mathbf{A}(0) \odot [\mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) - \mathbf{B}(0) \odot \mathbf{B}(0) \odot \mathbf{C}(0)] \right) \right\|_2 \\&\leq \sup_{t \in [0, T]} \|\mathbf{B}(t)\|_\infty^2 \|\mathbf{C}(t)\|_\infty \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) - \mathbf{A}(0)) \right\|_2 \\&+ \sup_{t \in [0, T]} \|\mathbf{A}(0)\|_\infty \left\| \frac{1}{\sqrt{N}} (\mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) - \mathbf{B}(0) \odot \mathbf{B}(0) \odot \mathbf{C}(0)) \right\|_2 \\&\lesssim \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) - \mathbf{A}(0)) \right\|_2 + \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{B}(t) \odot \mathbf{B}(t) \odot \mathbf{C}(t) - \mathbf{B}(0) \odot \mathbf{B}(0) \odot \mathbf{C}(0)) \right\|_2 \\&\dots \\&\lesssim \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{A}(t) - \mathbf{A}(0)) \right\|_2 + \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{B}(t) - \mathbf{B}(0)) \right\|_2 \\&+ \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{C}(t) - \mathbf{C}(0)) \right\|_2 + \sup_{t \in [0, T]} \left\| \frac{1}{\sqrt{N}} (\mathbf{D}(t) - \mathbf{D}(0)) \right\|_2 \\&\rightarrow 0,\end{aligned}$$

as $N \rightarrow \infty$. We can use the same strategy to I_2 as well as $\mathbf{W}^{(1)}$ and $\mathbf{b}^{(0)}$. As a consequence, we conclude

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u_{xx}(\mathbf{x}; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 = 0.$$

This concludes the proof. $\square$

With these lemmas, now we can prove our main Theorem 4.4

Proof of Theorem 4.4. For a given data set $\{x_b^i, g(x_b^i)\}_{i=1}^{N_b}, \{x_r^i, f(x_r^i)\}_{i=1}^{N_r}$, let $\mathbf{J}_u(t)$ and $\mathbf{J}_r(t)$ be the Jacobian matrix of $u(x_b; \boldsymbol{\theta}(t))$ and $u_{xx}(x_r; \boldsymbol{\theta})$ with respect to $\boldsymbol{\theta}$, respectively,

$$\mathbf{J}_u(t) = \left(\frac{\partial u(x_b^i; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}_j} \right), \mathbf{J}_r(t) = \left(\frac{\partial u_{xx}(x_r^i; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}_j} \right).$$

Note that

$$\mathbf{K}(t) = \begin{bmatrix} \mathbf{J}_u(t) \\ \mathbf{J}_r(t) \end{bmatrix} [\mathbf{J}_u^T(t), \mathbf{J}_r^T(t)] := \mathbf{J}(t) \mathbf{J}^T(t).$$

This implies that

$$\begin{aligned} \|\mathbf{K}(t) - \mathbf{K}(0)\|_2 &= \|\mathbf{J}(t) \mathbf{J}^T(t) - \mathbf{J}(0) \mathbf{J}^T(0)\|_2 \\ &\leq \|\mathbf{J}(t) [\mathbf{J}^T(t) - \mathbf{J}^T(0)]\|_2 + \|[\mathbf{J}(t) - \mathbf{J}(0)] \mathbf{J}^T(0)\|_2 \\ &\leq \|\mathbf{J}(t)\|_2 \|\mathbf{J}(t) - \mathbf{J}(0)\|_2 + \|\mathbf{J}(t) - \mathbf{J}(0)\|_2 \|\mathbf{J}(0)\|_2. \end{aligned}$$

By Lemma D.1, it is easy to show that $\|\mathbf{J}(t)\|_2$ is bounded. So it now suffices to show that

$$\sup_{t \in [0, T]} \|\mathbf{J}_u(t) - \mathbf{J}_u(0)\|_F \rightarrow 0 \quad (\text{D.6})$$

$$\sup_{t \in [0, T]} \|\mathbf{J}_r(t) - \mathbf{J}_r(0)\|_F \rightarrow 0, \quad (\text{D.7})$$

as $N \rightarrow \infty$. Since the training data is finite, it suffices to consider just two inputs x, x' . By the Cauchy-Schwartz inequality, we obtain

$$\begin{aligned} &\left| \left\langle \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\rangle - \left\langle \frac{\partial u(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\rangle \right| \\ &\leq \left| \left\langle \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\rangle \right| + \left| \left\langle \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\rangle \right| \\ &\leq \left\| \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\|_2 \left\| \frac{\partial u(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 + \left\| \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \left\| \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2. \end{aligned}$$

From Lemma D.4, we have $\left\| \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\|_2$ is uniformly bounded for $t \in [0, T]$. Then using Lemma D.4 again gives

$$\begin{aligned} &\sup_{t \in [0, T]} \left| \left\langle \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\rangle - \left\langle \frac{\partial u(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}}, \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\rangle \right| \\ &\leq C \sup_{t \in [0, T]} \left\| \frac{\partial u(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 + C \sup_{t \in [0, T]} \left\| \frac{\partial u(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \\ &\rightarrow 0, \end{aligned}$$

as $N \rightarrow \infty$. This implies that

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \|\mathbf{J}_u(t) - \mathbf{J}_u(0)\|_2 = 0.$$

Similarly, we can repeat this calculation for $\mathbf{J}_r$, i.e.,

$$\begin{aligned} &\sup_{t \in [0, T]} \left| \left\langle \frac{\partial u_{xx}(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}}, \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\rangle - \left\langle \frac{\partial u_{xx}(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}}, \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\rangle \right| \\ &\leq \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} \right\|_2 \left\| \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \\ &\quad + \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u_{xx}(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \left\| \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \\ &\leq C \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u_{xx}(x'; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 + C \sup_{t \in [0, T]} \left\| \frac{\partial u_{xx}(x; \boldsymbol{\theta}(t))}{\partial \boldsymbol{\theta}} - \frac{\partial u_{xx}(x; \boldsymbol{\theta}(0))}{\partial \boldsymbol{\theta}} \right\|_2 \\ &\rightarrow 0, \end{aligned}$$

as $N \rightarrow \infty$. Hence, we get

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \|J_r(t) - J_r(0)\|_2 = 0,$$

and thus we conclude that

$$\lim_{N \rightarrow \infty} \sup_{t \in [0, T]} \|K(t) - K(0)\|_2 = 0.$$

This concludes the proof. $\square$

References

- [1] Maziar Raissi, Alireza Yazdani, George Em Karniadakis, Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations, *Science* 367 (6481) (2020) 1026–1030.
- [2] Luning Sun, Han Gao, Shaowu Pan, Jian-Xun Wang, Surrogate modeling for fluid flows based on physics-constrained deep learning without simulation data, *Comput. Methods Appl. Mech. Eng.* 361 (2020) 112732.
- [3] Maziar Raissi, Hessam Babaei, Peyman Givi, Deep learning of turbulent scalar mixing, *Phys. Rev. Fluids* 4 (12) (2019) 124501.
- [4] Maziar Raissi, Zhicheng Wang, Michael S. Triantafyllou, George Em Karniadakis, Deep learning of vortex-induced vibrations, *J. Fluid Mech.* 861 (2019) 119–137.
- [5] Xiaowei Jin, Shengze Cai, Hui Li, George Em Karniadakis, NSFnets (Navier-Stokes flow nets): physics-informed neural networks for the incompressible Navier-Stokes equations, *arXiv preprint arXiv:2003.06496*, 2020.
- [6] Francisco Sahli Costabal, Yibo Yang, Paris Perdikaris, Daniel E. Hurtado, Ellen Kuhl, Physics-informed neural networks for cardiac activation mapping, *Front. Phys.* 8 (2020) 42.
- [7] Georgios Kissas, Yibo Yang, Eileen Hwuang, Walter R. Witschey, John A. Detre, Paris Perdikaris, Machine learning in cardiovascular flows modeling: predicting arterial blood pressure from non-invasive 4D flow MRI data using physics-informed neural networks, *Comput. Methods Appl. Mech. Eng.* 358 (2020) 112623.
- [8] Zhiwei Fang, Justin Zhan, Deep physical informed neural networks for metamaterial design, *IEEE Access* 8 (2019) 24506–24513.
- [9] Dehao Liu, Yan Wang, Multi-fidelity physics-constrained neural network and its application in materials modeling, *J. Mech. Des.* 141 (12) (2019).
- [10] Yuyao Chen, Lu Lu, George Em Karniadakis, Luca Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, *Opt. Express* 28 (8) (2020) 11618–11633.
- [11] Sifan Wang, Paris Perdikaris, Deep learning of free boundary and Stefan problems, *arXiv preprint, arXiv:2006.05311*, 2020.
- [12] Yibo Yang, Paris Perdikaris, Adversarial uncertainty quantification in physics-informed neural networks, *J. Comput. Phys.* 394 (2019) 136–152.
- [13] Yin hao Zhu, Nicholas Zabar, Phaedon-Stelios Koutsourelakis, Paris Perdikaris, Physics-constrained deep learning for high-dimensional surrogate modeling and uncertainty quantification without labeled data, *J. Comput. Phys.* 394 (2019) 56–81.
- [14] Yibo Yang, Paris Perdikaris, Physics-informed deep generative models, *arXiv preprint arXiv:1812.03511*, 2018.
- [15] Luning Sun, Jian-Xun Wang, Physics-constrained Bayesian neural network for fluid flow reconstruction with sparse and noisy data, *arXiv preprint arXiv:2001.05542*, 2020.
- [16] Liu Yang, Xuhui Meng, George Em Karniadakis, B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data, *arXiv preprint arXiv:2003.06097*, 2020.
- [17] Justin Sirignano, Konstantinos Spiliopoulos, DGM: a deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364.
- [18] Jiequn Han, Arnulf Jentzen, E. Weinan, Solving high-dimensional partial differential equations using deep learning, *Proc. Natl. Acad. Sci. USA* 115 (34) (2018) 8505–8510.
- [19] Dongkun Zhang, Ling Guo, George Em Karniadakis, Learning in modal space: solving time-dependent stochastic PDEs using physics-informed neural networks, *SIAM J. Sci. Comput.* 42 (2) (2020) A639–A665.
- [20] Guofei Pang, Lu Lu, George Em Karniadakis, fPINNs: fractional physics-informed neural networks, *SIAM J. Sci. Comput.* 41 (4) (2019) A2603–A2626.
- [21] Guofei Pang, Marta D'Elia, Michael Parks, George E. Karniadakis, nPINNs: nonlocal physics-informed neural networks for a parametrized nonlocal universal Laplacian operator. Algorithms and applications, *arXiv preprint arXiv:2004.04276*, 2020.
- [22] Alexandre M. Tartakovsky, Carlos Ortiz Marrero, Paris Perdikaris, Guzel D. Tartakovsky, David Barajas-Solano, Learning parameters and constitutive relationships with physics informed deep neural networks, *arXiv preprint arXiv:1808.03398*, 2018.
- [23] Lu Lu, Pengzhan Jin, George Em Karniadakis, DeepONet: learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators, *arXiv preprint, arXiv:1910.03193*, 2019.
- [24] A.M. Tartakovsky, C. Ortiz Marrero, Paris Perdikaris, G.D. Tartakovsky, D. Barajas-Solano, Physics-informed deep neural networks for learning parameters and constitutive relationships in subsurface flow problems, *Water Resour. Res.* 56 (5) (2020) e2019WR026731.
- [25] Yeonjong Shin, Jerome Darbon, George Em Karniadakis, On the convergence and generalization of physics informed neural networks, *arXiv preprint arXiv:2004.01806*, 2020.
- [26] Hamdi A. Tchelepi, Olga Fuks, Limitations of physics informed machine learning for nonlinear two-phase transport in porous media, *J. Mach. Learn. Model. Comput.* 1 (1) (2020).
- [27] Maziar Raissi, Deep hidden physics models: deep learning of nonlinear partial differential equations, *J. Mach. Learn. Res.* 19 (1) (2018) 932–955.
- [28] Sifan Wang, Yujun Teng, Paris Perdikaris, Understanding and mitigating gradient pathologies in physics-informed neural networks, *arXiv preprint arXiv:2001.04536*, 2020.
- [29] Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, Aaron Courville, On the spectral bias of neural networks, in: *International Conference on Machine Learning*, 2019, pp. 5301–5310.
- [30] Yuan Cao, Zhiying Fang, Yue Wu, Ding-Xuan Zhou, Quanquan Gu, Towards understanding the spectral bias of deep learning, *arXiv preprint arXiv:1912.01198*, 2019.
- [31] Matthew Tancik, Pratul P. Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T. Barron, Ren Ng, Fourier features let networks learn high frequency functions in low dimensional domains, *arXiv preprint arXiv:2006.10739*, 2020.
- [32] Ronen Basri, Meirav Galun, Amnon Geifman, David Jacobs, Yoni Kasten, Shira Kritchman, Frequency bias in neural networks for input of non-uniform density, *arXiv preprint arXiv:2003.04560*, 2020.
- [33] Arthur Jacot, Franck Gabriel, Clément Hongler, Neural tangent kernel: convergence and generalization in neural networks, in: *Advances in Neural Information Processing Systems*, 2018, pp. 8571–8580.
- [34] Greg Yang, Scaling limits of wide neural networks with weight sharing: Gaussian process behavior, gradient independence, and neural tangent kernel derivation, *arXiv preprint arXiv:1902.04760*, 2019.

- [35] Alexander G. de, G. Matthews, Mark Rowland, Jiri Hron, Richard E. Turner, Zoubin Ghahramani, Gaussian process behaviour in wide deep neural networks, arXiv preprint arXiv:1804.11271, 2018.
- [36] Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, Jascha Sohl-Dickstein, Deep neural networks as Gaussian processes, arXiv preprint arXiv:1711.00165, 2017.
- [37] David J.C. MacKay, Introduction to Gaussian Processes, NATO ASI Series F Computer and Systems Sciences, vol. 168, 1998, pp. 133–166.
- [38] Sanjeev Arora, Simon S. Du, Wei Hu, Zhiyuan Li, Russ R. Salakhutdinov, Ruosong Wang, On exact computation with an infinitely wide neural net, in: Advances in Neural Information Processing Systems, 2019, pp. 8141–8150.
- [39] Maziar Raissi, Paris Perdikaris, George E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
- [40] Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, Jeffrey Pennington, Wide neural networks of any depth evolve as linear models under gradient descent, in: Advances in Neural Information Processing Systems, 2019, pp. 8572–8583.
- [41] Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, Zheng Ma, Frequency principle: Fourier analysis sheds light on deep neural networks, arXiv preprint arXiv:1901.06523, 2019.
- [42] Basri Ronen, David Jacobs, Yoni Kasten, Shira Kritchman, The convergence rate of neural networks for learned functions of different frequencies, in: Advances in Neural Information Processing Systems, 2019, pp. 4761–4771.
- [43] Lu Lu, Xuhui Meng, Zhiping Mao, George E. Karniadakis, DeepXDE: a deep learning library for solving differential equations, arXiv preprint arXiv:1907.04502, 2019.
- [44] Parviz Moin, Fundamentals of Engineering Numerical Analysis, Cambridge University Press, 2010.
- [45] L.C. Evans, American Mathematical Society. Partial Differential Equations, Graduate Studies in Mathematics, American Mathematical Society, 1998.
- [46] Xavier Glorot, Yoshua Bengio, Understanding the difficulty of training deep feedforward neural networks, in: Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, 2010, pp. 249–256.
- [47] Diederik P. Kingma, Jimmy Ba Adam, A method for stochastic optimization, arXiv preprint arXiv:1412.6980, 2014.
- [48] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, Andrew Rabinovich, GradNorm: gradient normalization for adaptive loss balancing in deep multitask networks, in: International Conference on Machine Learning, PMLR, 2018, pp. 794–803.
- [49] Haowen Xu, Hao Zhang, Zhiting Hu, Xiaodan Liang, Ruslan Salakhutdinov, Eric Xing, AutoLoss: learning discrete schedules for alternate optimization, arXiv preprint arXiv:1810.02442, 2018.
- [50] A. Ali Heydari, Craig A. Thompson, Asif Mehmood, SoftAdapt: techniques for adaptive loss weighting of neural networks with multi-part loss functions, arXiv preprint arXiv:1912.12355, 2019.